

Méthodologie ASIC numérique

De SystemVerilog au GDS avec Xcelium, Genus et Innovus

Karim Sabra

Parcours progressif — niveau initial à avancé
16 juillet 2026

Scripts sous licence MIT ; texte sous CC BY 4.0.

Préface et contrat de lecture

Ce manuel n'est ni une copie d'un manuel Cadence, ni une distribution de PDK. Il explique une méthodologie progressive à partir de commandes courtes, puis de scripts réutilisables dont chaque entrée, sortie et condition d'arrêt est visible. Le lecteur doit disposer séparément des licences et collatéraux autorisés par son établissement.

Le fil conducteur utilise d'abord un full adder combinatoire, puis un additionneur 8 bits pipeliné. Cette progression empêche une erreur pédagogique fréquente : montrer un arbre d'horloge sur un circuit qui ne possède aucune horloge. Les termes EDA officiels restent en anglais et sont expliqués en français à leur première apparition.

Chaque outil est présenté dans le même ordre :

1. comprendre les fichiers minimaux ;
2. exécuter une commande directe ;
3. lire le script ligne par ligne ;
4. ajouter les contrôles et les vues avancées.

Frontière de preuve

Un retour outil égal à zéro indique seulement que le processus s'est terminé. Il ne prouve ni la couverture des contraintes, ni la connectivité PG, ni le DRC/LVS, ni le signoff. Chaque chapitre définit donc ses propres gates.

Portée et statuts

La portée exécutable va du RTL au GDS candidat : simulation RTL, synthèse, floorplan, power planning, placement, CTS lorsque applicable, routage, extraction, STA et exports.

Les statuts finaux possibles sont séparés :

Statut	Sens
PASS	La gate nommée possède la preuve exigée.
FAIL	Le run a produit une preuve négative ou incomplète.
NOT_APPLICABLE	L'étape n'a pas de sens pour ce design.
PROVISIONAL	L'implémentation peut être étudiée, mais un propriétaire ou un outil externe doit encore qualifier l'hypothèse.
DEFERRED	La vérification appartient explicitement à une étape future.

Dans cette édition, `SIGNOFF_STATUS=NOT_ACHIEVED` reste obligatoire : PVS DRC/LVS, qualification tap/endcap, IR/EM et tapeout review ne font pas partie des preuves exécutables.

Table des matières

Préface et contrat de lecture	i
Portée et statuts	ii
1 Vue d'ensemble du flow numérique ASIC	1
1.1 Les trois outils du parcours	1
1.2 Même progression pour chaque outil	1
1.3 Artefacts essentiels	1
1.4 Pourquoi deux exemples	2
2 Préparer l'environnement et les fichiers	3
2.1 Arborescence volontairement courte	3
2.2 Configuration privée du laboratoire	3
2.3 D'abord la commande, ensuite le wrapper	4
2.4 Checklist avant le premier lancement	4
3 RTL du full adder et additionneur pipeliné	5
3.1 Contrat du full adder	5
3.2 Disciplines de codage	6
3.3 Construire un datapath séquentiel sans changer de sujet	6
3.4 Latence et protocole valid	7
4 Xcelium : du testbench minimal au script réutilisable	8
4.1 Niveau 1 — un testbench simple et auto-vérifiant	8
4.2 Niveau 2 — séparer RTL et testbench avec des filelists	9
4.3 Niveau 3 — comprendre le script réutilisable	9
4.4 Niveau 4 — augmenter la complexité avec le pipeline	12
5 Waveforms et méthode de debug	14
5.1 Batch d'abord, GUI ensuite	14
5.2 Lire le full adder	14
5.3 Lire le pipeline	14
5.4 Ordre de diagnostic	14
6 Contraintes SDC et analyse temporelle statique	15
6.1 Le SDC est une spécification	15
6.2 Horloge virtuelle du full adder	15
6.3 Horloge réelle du pipeline	16
6.4 Setup et hold	16
6.5 Couverture des contraintes	16

7	Genus : du Tcl minimal au flow physique	17
7.1	Niveau 1 — les entrées minimales	17
7.2	Niveau 2 — comprendre les trois transformations	18
7.3	Niveau 3 — utiliser le wrapper	18
7.4	Niveau 4 — conscience physique et MMMC	22
7.5	Rapports à lire	25
8	Innovus : de l'import minimal au floorplan contrôlé	27
8.1	Niveau 1 — fichiers nécessaires	27
8.2	Niveau 2 — Tcl minimal	27
8.3	Niveau 3 — préflight et wrapper	29
8.4	Niveau 4 — import avancé et floorplan	33
8.5	Pins déterministes	34
9	Power planning, lignes et placement	35
9.1	Deux vocabulaires pour VDD et VSS	35
9.2	Anneau et rails	35
9.3	Infrastructure de lignes JIHD	35
9.4	Placement et optimisation pré-CTS	35
10	CTS, routage et optimisation post-route	37
10.1	CTS conditionnel	37
10.2	Optimisation post-CTS	37
10.3	Routage des signaux	37
10.4	Post-route	37
10.5	Critères finaux de route	38
11	Extraction, STA final et exports	39
11.1	Extraction RC	39
11.2	STA final	39
11.3	Livrables du niveau avancé	39
11.4	Pourquoi le GDS reste un candidat	39
12	Gates et diagnostic	40
12.1	Séparer exécution et qualité	40
12.2	Chercher le premier échec causal	40
12.3	Méthode en cinq questions	40
12.4	Ordre de lecture conseillé	40
13	Méthodologie avancée et scripts réutilisables	41
13.1	La règle d'escalade	41
13.2	Simulation : simple puis avancée	41
13.3	Synthèse : logique minimale puis physique	41
13.4	Implémentation : démonstration puis flow complet	42
13.5	Architecture recommandée des scripts	42
13.6	Critère d'arrêt	42
A	Commandes de référence	43
A.1	Simulation Xcelium	43
A.2	Synthèse Genus	43
A.3	Implémentation Innovus	43
A.4	Inspection après un run	44

B Erreurs courantes	45
C Glossaire	46
Références	47

Chapitre 1

Vue d'ensemble du flow numérique ASIC

Une implémentation numérique transforme une description RTL en géométrie manufacturable. Les outils changent la représentation du design, mais ils ne doivent pas changer son contrat fonctionnel.

1.1 Les trois outils du parcours

1. **Xcelium** compile, élabore et simule le RTL avec son testbench.
2. **Genus** lit le même RTL, applique le SDC et mappe la logique vers une bibliothèque de cellules standard.
3. **Innovus** importe la netlist et les vues physiques, construit le floorplan, place les cellules, traite l'horloge si elle existe, route les nets et extrait les parasites.

La simulation répond à des questions fonctionnelles. Le STA répond à des questions temporelles. Les vérifications physiques répondent aux questions de géométrie et de connectivité. Ces preuves sont complémentaires.

1.2 Même progression pour chaque outil

Niveau	Objectif
1 — minimal	Identifier les fichiers indispensables et lancer une commande courte, visible et compréhensible.
2 — séparé	Placer RTL, testbench, filelists, contraintes et paramètres dans des fichiers dédiés.
3 — réutilisable	Employer un script qui valide les entrées, crée un dossier de résultats et conserve le code retour de l'outil.
4 — avancé	Ajouter MMMC, contrôles de qualité, étapes physiques, critères d'arrêt et rapports structurés.

1.3 Artefacts essentiels

Artefact	Producteur	Rôle
RTL et filelist	concepteur	Décrire le comportement et l'ordre de lecture.

Testbench SDC	vérification concepteur timing	Appliquer les stimuli et rendre un verdict. Définir clocks, délais I/O, charges et exceptions.
Liberty	technologie	Décrire timing, puissance et règles électriques.
LEF	technologie	Décrire sites, couches et géométrie abstraite.
Netlist mappée	Genus	Réaliser la logique avec des cellules standard.
MMMC	flow timing	Associer modes, bibliothèques et corners RC.
DEF et checkpoint	Innovus	Conserver l'état physique du bloc.
SPEF	Innovus	Exporter les parasites extraits.
GDS	Innovus	Exporter une géométrie candidate pour vérification.

1.4 Pourquoi deux exemples

Le full adder 1 bit possède huit combinaisons et aucune horloge. Il permet d'apprendre le RTL, le testbench, les filelists, une contrainte combinatoire, la synthèse et un premier placement sans mélanger les notions.

L'additionneur pipeliné ajoute des registres, une clock, un reset et un signal de validité. Il permet ensuite d'étudier latence, setup, hold, MMMC et CTS.

À retenir

Commencer petit permet de distinguer une erreur de méthode d'une difficulté réelle du design.
Une étape sans objet, comme le CTS du full adder, doit être marquée `NOT_APPLICABLE`.

Les commandes Cadence sont fondées sur les références installées [1, 2, 3]. Leur disponibilité et leur syntaxe exacte doivent être confirmées dans l'environnement du laboratoire.

Chapitre 2

Préparer l'environnement et les fichiers

2.1 Arborescence volontairement courte

Le matériel pédagogique utilise quatre ensembles :

examples/ RTL, testbenches, filelists, SDC et paramètres des deux exemples.

scripts/ trois lanceurs shell : simulation, synthèse et implémentation.

flow/ scripts Tcl lus par Genus et Innovus.

config/ modèle des variables technologiques à adapter dans un espace privé du laboratoire.

Les résultats sont placés par défaut sous `/tmp/digi_tuto_runs`. Le dossier contenant le guide reste donc composé uniquement des sources utiles et des PDF.

2.2 Configuration privée du laboratoire

Listing 2.1 – Modèle de configuration Xcelium, Genus et Innovus

```
1  #!/usr/bin/env bash
2  # Enregistrez une copie de ce modèle dans un espace privé du laboratoire,
3  # adaptez les chemins puis indiquez-la avec LAB_CONFIG=/chemin/prive/lab.env.
4
5  # Initialisation et exécutables Cadence
6  CADENCE_SETUP=/chemin/vers/le/setup_cadence
7  XRUN_BIN=xrun
8  GENUS_BIN=genus
9  INNOVUS_BIN=innovus
10
11 # Profil technologique de référence
12 TECHNOLOGY_NAME=XH018
13 TECHNOLOGY_STACK=xx31
14 STDCELL_FAMILY=JIHD
15 STDCELL_SITE=core_jihd
16 SIGNAL_BOTTOM_LAYER=MET1
17 SIGNAL_TOP_LAYER=MET3
18 POWER_HORIZONTAL_LAYER=MET3
19 POWER_VERTICAL_LAYER=METTP
20 STDCELL_POWER_PIN=vddi
21 STDCELL_GROUND_PIN=gndi
22
23 # Genus. Les listes Liberty et LEF utilisent ':' comme séparateur.
24 GENUS_LIBERTY_TC=/chemin/vers/la/liberty_typique
25 GENUS_LIBERTY_BC=/chemin/vers/la/liberty_rapide
26 GENUS_LIBERTY_WC=/chemin/vers/la/liberty_lente
27 GENUS_LEF_FILES=/chemin/vers/tech.lef:/chemin/vers/stdcell.lef
28 GENUS_QRC_TC=/chemin/vers/qrc_typique
29 GENUS_QRC_BC=/chemin/vers/qrc_min
30 GENUS_QRC_WC=/chemin/vers/qrc_max
31 GENUS_TEMPERATURE_TC=25
32 GENUS_TEMPERATURE_BC=-40
```

```

33 GENUS_TEMPERATURE_WC=125
34
35 # Innovus
36 INNOVUS_TECH_LEF=/chemin/vers/tech.lef
37 INNOVUS_STDCELL_LEF=/chemin/vers/stdcell.lef
38 INNOVUS_LIBERTY_TC=/chemin/vers/la/liberty_typique
39 INNOVUS_LIBERTY_BC=/chemin/vers/la/liberty_rapide
40 INNOVUS_LIBERTY_WC=/chemin/vers/la/liberty_lente
41 INNOVUS_QRC_TC=/chemin/vers/qrc_typique
42 INNOVUS_QRC_BC=/chemin/vers/qrc_min
43 INNOVUS_QRC_WC=/chemin/vers/qrc_max
44 INNOVUS_STREAM_MAP=/chemin/vers/la/stream_map
45 INNOVUS_STDCELL_GDS=/chemin/vers/le/gds_standard_cells
46
47 # Cellules physiques: chaque valeur est une liste séparée par des espaces.
48 INNOVUS_FILLER_CELLS="FEED25JIHD FEED15JIHD FEED10JIHD FEED7JIHD FEED5JIHD FEED3JIHD FEED2JIHD FEED1JIHD"
49 INNOVUS_TIE_HIGH_CELLS="LOGIC1DJJHD LOGIC1LVJJHD"
50 INNOVUS_TIE_LOW_CELLS="LOGICODJJHD LOGICOLVJJHD"
51 INNOVUS_ANTENNA_CELLS="ANTENNACELLN2JIHD ANTENNACELLP2JIHD ANTENNACELLP2JIHD"
52 INNOVUS_CTS_BUFFERS="BUJIHDX1 BUJIHDX2 BUJIHDX3 BUJIHDX4 BUJIHDX6 BUJIHDX8 BUJIHDX12"
53 INNOVUS_CTS_INVERTERS="INJIHDX0 INJIHDX1 INJIHDX2 INJIHDX3 INJIHDX4 INJIHDX6 INJIHDX8 INJIHDX12"
54
55 # Floorplan pédagogique. Les paramètres PG doivent être confirmés dans les
56 # règles du PDK avant le premier run réel.
57 CORE_UTILIZATION=0.60
58 CORE_ASPECT_RATIO=1.0
59 CORE_MIN_SIDE_UM=60.0
60 CORE_MARGIN_UM=20.0
61 PG_RING_WIDTH_UM=2.0
62 PG_RING_SPACING_UM=1.0
63 PG_RING_OFFSET_UM=2.0

```

Le modèle ne contient que des noms de variables. Une copie privée fournit les chemins réels vers Liberty, LEF, QRC, stream map, GDS de cellules et script d'initialisation Cadence :

```

1 cp config/technology.example.env /chemin/prive/lab.env
2 # Editor /chemin/prive/lab.env, puis:
3 export LAB_CONFIG=/chemin/prive/lab.env

```

Les chemins technologiques ne doivent jamais être écrits dans les scripts génériques. Cette séparation permet de réutiliser exactement la même méthode sur une autre machine en changeant seulement LAB_CONFIG.

2.3 D'abord la commande, ensuite le wrapper

Chaque chapitre montre d'abord l'appel direct à l'outil. Cette forme est idéale pour apprendre, car aucune étape n'est cachée. Le wrapper devient utile lorsque la commande fonctionne : il contrôle les entrées, choisit un dossier de résultats unique et vérifie les sorties attendues.

Ne pas automatiser une erreur

Un script plus long ne corrige pas une mauvaise filelist, un top incorrect ou un SDC incomplet. La commande minimale doit être comprise et validée avant d'ajouter l'automatisation.

2.4 Checklist avant le premier lancement

1. charger l'environnement Cadence autorisé ;
2. vérifier `command -v xrun`, `command -v genus` et `command -v innovus` ;
3. ouvrir les filelists et confirmer que chaque chemin existe ;
4. confirmer le nom exact du top RTL et du top testbench ;
5. renseigner LAB_CONFIG avant Genus ou Innovus ;
6. lancer le niveau minimal avant le niveau avancé.

Chapitre 3

RTL du full adder et additionneur pipeliné

3.1 Contrat du full adder

Pour trois bits d'entrée a , b et c_{in} , le résultat arithmétique possède deux bits : une somme et une retenue. Les équations sont

$$sum = a \oplus b \oplus c_{in}, \quad (3.1)$$

$$c_{out} = ab + ac_{in} + bc_{in}. \quad (3.2)$$

a	b	c_{in}	sum	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Le RTL encode explicitement ces équations. Cette forme est plus pédagogique qu'une addition comportementale et permet de reconnaître la logique dans le schéma Genus.

Listing 3.1 – RTL synthétisable du full adder

```
1  'timescale 1ns/1ps
2
3  // Additionneur complet combinatoire sur un bit.
4  //
5  // Les equations sont ecrites explicitement afin que le lien entre la table
6  // de verite, le RTL et les portes obtenues apres synthese reste visible.
7  module full_adder_comb (
8      input  logic a_i,
9      input  logic b_i,
10     input  logic cin_i,
11     output logic sum_o,
12     output logic cout_o
13 );
14
15     assign sum_o  = a_i ^ b_i ^ cin_i;
16     assign cout_o = (a_i & b_i) | (a_i & cin_i) | (b_i & cin_i);
17
18 endmodule
```

3.2 Disciplines de codage

`default_nettype none` transforme une faute de frappe en erreur de compilation au lieu de créer un net implicite. Les ports utilisent `logic` et le design ne contient aucun délai, initialisation de simulation ou appel système. Il est donc synthétisable et indépendant du testbench [4].

Le fichier restaure `default_nettype wire` après le module afin de ne pas imposer une directive globale aux fichiers tiers compilés ensuite.

3.3 Construire un datapath séquentiel sans changer de sujet

La chaîne ripple-carry instancie *WIDTH* full adders. Une banque de registres capture *a*, *b*, *c_{in}* et *valid*. La logique combinatoire calcule le résultat pendant un cycle. La banque de sortie capture somme, retenue et validité au front suivant.

Listing 3.2 – Additionneur ripple-carry pipeline

```

1  'timescale 1ns/1ps
2
3  // Additionneur paramétrique avec registres d'entrée et de sortie.
4  //
5  // Une transaction capturée sur un front montant est présentée sur les sorties
6  // au front montant suivant : la latence fonctionnelle vaut donc un cycle.
7  module adder_pipeline #(
8      parameter int unsigned WIDTH = 8
9  ) (
10     input logic      clk_i,
11     input logic      rst_ni,
12     input logic      valid_i,
13     input logic [WIDTH-1:0] a_i,
14     input logic [WIDTH-1:0] b_i,
15     input logic      cin_i,
16     output logic      valid_o,
17     output logic [WIDTH-1:0] sum_o,
18     output logic      cout_o
19 );
20
21     logic [WIDTH-1:0] a_q;
22     logic [WIDTH-1:0] b_q;
23     logic      cin_q;
24     logic      valid_q;
25
26     logic [WIDTH-1:0] sum_comb;
27     logic [WIDTH:0]   carry_comb;
28
29     assign carry_comb[0] = cin_q;
30
31     // La chaîne de retenue instancie le même full adder que le premier exemple.
32     for (genvar bit_index = 0; bit_index < WIDTH; bit_index++) begin : gen_full_adders
33         full_adder_comb u_full_adder (
34             .a_i    (a_q[bit_index]),
35             .b_i    (b_q[bit_index]),
36             .cin_i   (carry_comb[bit_index]),
37             .sum_o   (sum_comb[bit_index]),
38             .cout_o  (carry_comb[bit_index + 1])
39         );
40     end
41
42     // Reset synchrone, actif à l'état bas.
43     always_ff @(posedge clk_i) begin
44         if (!rst_ni) begin
45             a_q    <= '0;
46             b_q    <= '0;
47             cin_q   <= 1'b0;
48             valid_q <= 1'b0;
49             valid_o <= 1'b0;
50             sum_o   <= '0;
51             cout_o  <= 1'b0;
52         end else begin
53             a_q    <= a_i;

```

```
54      b_q    <= b_i;
55      cin_q  <= cin_i;
56      valid_q <= valid_i;
57
58      valid_o <= valid_q;
59      if (valid_q) begin
60          sum_o  <= sum_comb;
61          cout_o <= carry_comb[WIDTH];
62      end else begin
63          sum_o  <= '0;
64          cout_o <= 1'b0;
65      end
66  end
67 end
68
69 endmodule
```

Le reset est synchrone actif bas. Il est évalué uniquement au front montant et reste donc un chemin synchrone contraint. Cette décision garde le premier flow centré sur setup/hold; recovery/removal et RDC appartiennent à une extension future consacrée aux resets asynchrones.

3.4 Latence et protocole valid

Une transaction est acceptée quand `valid_i` vaut un au front montant. Le résultat correspondant devient valide au front suivant. Les données de sortie n'ont de sens contractuel que lorsque `valid_o` vaut un. Cette règle permet au testbench d'insérer des bulles sans confondre une ancienne valeur de sortie avec une nouvelle transaction.

Largeur et cast

La référence du testbench doit calculer sur $WIDTH + 1$ bits. Une addition limitée à $WIDTH$ bits perdrait précisément la retenue que le design doit vérifier.

Chapitre 4

Xcelium : du testbench minimal au script réutilisable

4.1 Niveau 1 — un testbench simple et auto-vérifiant

Le full adder possède seulement huit combinaisons. Le testbench applique chacune d'elles, calcule la référence avec les mêmes équations booléennes, compare avec `!=` afin de détecter aussi les valeurs inconnues, puis termine par un marqueur clair.

Listing 4.1 – Testbench exhaustif du full adder

```
1  'timescale 1ns/1ps
2
3  // Banc de test dirige et auto-verifiant du full adder.
4  module tb_full_adder_comb;
5
6      logic a_i;
7      logic b_i;
8      logic cin_i;
9      logic sum_o;
10     logic cout_o;
11
12     full_adder_comb dut (
13         .a_i    (a_i),
14         .b_i    (b_i),
15         .cin_i  (cin_i),
16         .sum_o  (sum_o),
17         .cout_o (cout_o)
18     );
19
20     initial begin : test_exhaustif
21         logic sum_attendue;
22         logic cout_attendue;
23
24         a_i    = 1'b0;
25         b_i    = 1'b0;
26         cin_i  = 1'b0;
27
28         // Les huit combinaisons de la table de verite sont testees.
29         for (int vecteur = 0; vecteur < 8; vecteur++) begin
30             {a_i, b_i, cin_i} = vecteur[2:0];
31             // Le delai laisse le temps aux mises a jour combinatoires.
32             #10;
33
34             sum_attendue = a_i ^ b_i ^ cin_i;
35             cout_attendue = (a_i & b_i) | (a_i & cin_i) | (b_i & cin_i);
36
37             if ((sum_o != sum_attendue) || (cout_o != cout_attendue)) begin
38                 $fatal(1,
39                     "Echec vecteur %03b : obtenu cout,sum=%b%b, attendu=%b%b",
40                     {a_i, b_i, cin_i}, cout_o, sum_o,
41                     cout_attendue, sum_attendue);
42             end
43         end
```

```

44         $display("TEST_PASS: full_adder_comb");
45         $finish;
46     end
47 end
48
49 endmodule

```

Un testbench utile doit rendre un verdict sans inspection manuelle : `$fatal` en cas d'erreur et `TEST_PASS` uniquement lorsque tous les vecteurs ont réussi.

4.2 Niveau 2 — séparer RTL et testbench avec des filelists

La filelist RTL contient :

```

1 examples/01_full_adder_comb/rtl/full_adder_comb.sv

```

La filelist testbench contient :

```

1 examples/01_full_adder_comb/tb/tb_full_adder_comb.sv

```

Depuis le dossier principal du tutoriel, la commande minimale est :

```

1 xrun -64bit -sv -timescale 1ns/1ps \
2   -f examples/01_full_adder_comb/sim/rtl.f \
3   -f examples/01_full_adder_comb/sim/tb.f \
4   -top tb_full_adder_comb -R

```

`xrun` compile d'abord les fichiers, élabore `tb_full_adder_comb`, puis exécute la simulation avec `-R`. L'ordre RTL puis testbench évite qu'une définition nécessaire soit découverte trop tard dans des configurations strictes.

4.3 Niveau 3 — comprendre le script réutilisable

Listing 4.2 – Lanceur Xcelium réutilisable

```

1  #!/usr/bin/env bash
2  # Simulation RTL Xcelium, du full adder au pipeline.
3
4  set -Eeuo pipefail
5
6  usage() {
7      cat <<'EOF'
8  Usage:
9      bash scripts/run_sim.sh full_adder_comb [--gui]
10     bash scripts/run_sim.sh adder_pipeline [--gui]
11
12 Variables optionnelles:
13     XRUN_BIN=xrun
14     RUN_ROOT=/tmp/digi_tuto_runs
15     RUN_ID=nom_du_run
16     XRUN_EXTRA_ARGS="..."
17 EOF
18 }
19
20 script_dir="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
21 project_root="$(cd -- "${script_dir}/.." && pwd -P)"
22
23 design=""
24 gui=0
25 while (($# > 0)); do
26     case "$1" in
27         --gui)
28             gui=1
29             shift
30             ;;
31         -h|--help)
32             usage

```

```

33         exit 0
34     ;;
35     full_adder_comb|adder_pipeline)
36     [[ -z "$design" ]] || {
37         printf 'ERREUR: un seul design est attendu\n' >&2
38         exit 2
39     }
40     design=$1
41     shift
42     ;;
43     *)
44         printf 'ERREUR: argument inconnu: %s\n' "$1" >&2
45         usage >&2
46         exit 2
47     ;;
48     esac
49 done
50
51 case "$design" in
52     full_adder_comb)
53         example_dir="$project_root/examples/01_full_adder_comb"
54         tb_top=tb_full_adder_comb
55         pass_marker="TEST_PASS: full_adder_comb"
56         ;;
57     adder_pipeline)
58         example_dir="$project_root/examples/02_adder_pipeline"
59         tb_top=tb_adder_pipeline
60         pass_marker="TEST_PASS: adder_pipeline"
61         ;;
62     *)
63         printf 'ERREUR: design absent\n' >&2
64         usage >&2
65         exit 2
66     ;;
67 esac
68
69 xrun_bin=${XRUN_BIN:-xrun}
70 if ! command -v "$xrun_bin" >/dev/null 2>&1; then
71     printf 'ERREUR: xrun est introuvable; chargez l environnement Cadence\n' >&2
72     exit 127
73 fi
74
75 run_root=${RUN_ROOT:-/tmp/digi_tuto_runs}
76 run_id=${RUN_ID:-$(date -u +%Y%m%dT%H%M%SZ)}
77 if [[ ! "$run_id" =~ ^[A-Za-z0-9][A-Za-z0-9-_.]*$ ]]; then
78     printf 'ERREUR: RUN_ID invalide: %s\n' "$run_id" >&2
79     exit 2
80 fi
81 run_dir="$run_root/xcelium/$design/$run_id"
82 if [[ -e "$run_dir" ]]; then
83     printf 'ERREUR: le dossier existe déjà: %s\n' "$run_dir" >&2
84     exit 2
85 fi
86 mkdir -p "$run_dir"
87
88 resolve_filelist() {
89     local source_file=$1
90     local target_file=$2
91     local line
92     : > "$target_file"
93     while IFS= read -r line || [[ -n "$line" ]]; do
94         line=${line%#*}
95         line=${line#"${line%[![:space:]]*}"}
96         line=${line%"${line##*[![:space:]]}"}
97         [[ -n "$line" ]] || continue
98         if [[ "$line" == /* || "$line" == +* || "$line" == -* ]]; then
99             printf '%s\n' "$line" >> "$target_file"
100         else
101             printf '%s/%s\n' "$project_root" "$line" >> "$target_file"
102         fi
103     done < "$source_file"
104 }
105
106 resolve_filelist "$example_dir/sim/rtl.f" "$run_dir/rtl.f"

```

```

107 resolve_filelist "$example_dir/sim/tb.f" "$run_dir/tb.f"
108
109 tcl_file="$run_dir/run.tcl"
110 {
111     printf 'database -open waves -into {%s} -default\n' "$run_dir/waves.shm"
112     printf 'probe -create %s -all -depth all\n' "$tb_top"
113     printf 'run\n'
114     if ((gui == 0)); then
115         printf 'exit\n'
116     fi
117 } > "$tcl_file"
118
119 cmd=(
120     "$xrun_bin"
121     -64bit
122     -sv
123     -timescale 1ns/1ps
124     -access +rwc
125     -f "$run_dir/rtl.f"
126     -f "$run_dir/tb.f"
127     -top "$tb_top"
128     -xmlbdirname "$run_dir/xcelium.d"
129     -input "$tcl_file"
130     -logfile "$run_dir/xrun.log"
131 )
132 if ((gui == 1)); then
133     cmd+=(-gui)
134 fi
135 if [[ -n ${XRUN_EXTRA_ARGS:-} ]]; then
136     read -r -a extra_args <<< "$XRUN_EXTRA_ARGS"
137     cmd+=("${extra_args[@]}")
138 fi
139
140 printf 'Simulation Xcelium: %s\n' "$design"
141 printf 'Résultats: %s\n' "$run_dir"
142 set +e
143 "${cmd[@]}"
144 tool_rc=$?
145 set -e
146
147 if ((tool_rc != 0)); then
148     printf 'RESULTAT: FAIL (xrun=%d)\n' "$tool_rc" >&2
149     exit "$tool_rc"
150 fi
151 if ! grep -Fxq "$pass_marker" "$run_dir/xrun.log"; then
152     printf 'RESULTAT: FAIL (marqueur final absent)\n' >&2
153     exit 1
154 fi
155
156 printf 'RESULTAT: PASS\n'

```

Le script réalise les opérations suivantes :

1. identifier l'exemple et le top testbench ;
2. vérifier que xrun est disponible ;
3. créer un dossier unique sous /tmp/digi_tuto_runs/xcelium/ ;
4. convertir les chemins des filelists en chemins absolus ;
5. isoler la base compilée avec -xmlbdirname ;
6. enregistrer le log et les waveforms SHM ;
7. conserver le code retour de xrun ;
8. exiger le marqueur TEST_PASS.

Le lancement simple devient :

```
1 bash scripts/run_sim.sh full_adder_comb
```

Pour ouvrir l'interface graphique avec la même préparation :

```
1 bash scripts/run_sim.sh full_adder_comb --gui
```

4.4 Niveau 4 — augmenter la complexité avec le pipeline

L'additionneur pipeliné introduit clock, reset, latence et protocole `valid`. Sa filelist RTL place le full adder avant le module pipeline. Le scoreboard mémorise le résultat attendu pendant un cycle et vérifie la sortie après les mises à jour non bloquantes.

Listing 4.3 – Testbench du pipeline avec scoreboard

```

1  'timescale 1ns/1ps
2
3  // Banc de test dirige et auto-verifiant.
4  module tb_adder_pipeline;
5
6      localparam int unsigned WIDTH = 8;
7      localparam int unsigned NOMBRE_TRANSACTIONS = 8;
8
9      logic          clk_i;
10     logic          rst_ni;
11     logic          valid_i;
12     logic [WIDTH-1:0] a_i;
13     logic [WIDTH-1:0] b_i;
14     logic          cin_i;
15     logic          valid_o;
16     logic [WIDTH-1:0] sum_o;
17     logic          cout_o;
18
19     logic          valid_attendu_q = 1'b0;
20     logic [WIDTH:0] resultat_attendu_q = '0;
21     integer        transactions_verifiees = 0;
22
23     adder_pipeline #(
24         .WIDTH (WIDTH)
25     ) dut (
26         .clk_i   (clk_i),
27         .rst_ni  (rst_ni),
28         .valid_i (valid_i),
29         .a_i     (a_i),
30         .b_i     (b_i),
31         .cin_i   (cin_i),
32         .valid_o (valid_o),
33         .sum_o   (sum_o),
34         .cout_o  (cout_o)
35     );
36
37     initial begin
38         clk_i = 1'b0;
39         forever #5 clk_i = ~clk_i;
40     end
41
42     // Le scoreboard modele exactement la latence d'un cycle.
43     // La verification est retardee d'une nanoseconde apres le front actif afin
44     // d'observer les mises a jour non bloquantes.
45     always @(posedge clk_i) begin : scoreboard
46         if (!rst_ni) begin
47             #1;
48             if ((valid_o !== 1'b0) || (sum_o !== '0) || (cout_o !== 1'b0)) begin
49                 $fatal(1, "Le reset synchrone n'a pas remis les sorties a zero");
50             end
51             valid_attendu_q = 1'b0;
52             resultat_attendu_q = '0;
53         end else begin
54             #1;
55             if (valid_o !== valid_attendu_q) begin
56                 $fatal(1, "valid_o=%b, valeur attendue=%b",
57                     valid_o, valid_attendu_q);
58             end
59
60             if (valid_attendu_q) begin
61                 if ({cout_o, sum_o} !== resultat_attendu_q) begin
62                     $fatal(1,
63                         "Resultat obtenu=%h, attendu=%h",
64                         {cout_o, sum_o}, resultat_attendu_q);
65                 end

```

```

66         transactions_verifiees = transactions_verifiees + 1;
67     end else if ((sum_o !== '0') || (cout_o !== 1'b0)) begin
68         $fatal(1, "Les donnees de sortie doivent etre nulles sans valid_o");
69     end
70
71     valid_attendu_q = valid_i;
72     resultat_attendu_q = {1'b0, a_i}
73                         + {1'b0, b_i}
74                         + {{WIDTH{1'b0}}, cin_i};
75
76 end
77
78 // Les entrees sont appliquees au front descendant, a 5 ns du front actif.
79 task automatic appliquer_vecteur(
80     input logic          valid,
81     input logic [WIDTH-1:0] a,
82     input logic [WIDTH-1:0] b,
83     input logic          cin
84 );
85     @(negedge clk_i);
86     valid_i = valid;
87     a_i     = a;
88     b_i     = b;
89     cin_i   = cin;
90 endtask
91
92 initial begin : scenario_dirige
93     rst_ni = 1'b0;
94     valid_i = 1'b0;
95     a_i     = '0;
96     b_i     = '0;
97     cin_i   = 1'b0;
98
99     // Deux fronts actifs garantissent l'application du reset synchrone.
100    repeat (2) @(negedge clk_i);
101    rst_ni = 1'b1;
102
103    appliquer_vecteur(1'b1, 8'h00, 8'h00, 1'b0); // zero
104    appliquer_vecteur(1'b1, 8'hFF, 8'h00, 1'b1); // retenue de bout en bout
105    appliquer_vecteur(1'b0, 8'h00, 8'h00, 1'b0); // bulle de validite
106    appliquer_vecteur(1'b1, 8'h0F, 8'h01, 1'b0); // propagation locale
107    appliquer_vecteur(1'b1, 8'h55, 8'hAA, 1'b0); // motifs alternes
108    appliquer_vecteur(1'b1, 8'hA5, 8'h5A, 1'b1); // depassement avec cin
109    appliquer_vecteur(1'b0, 8'h12, 8'h34, 1'b0); // seconde bulle
110    appliquer_vecteur(1'b1, 8'h7F, 8'h01, 1'b0); // frontiere signee
111    appliquer_vecteur(1'b1, 8'h80, 8'h80, 1'b0); // retenue de poids fort
112    appliquer_vecteur(1'b1, 8'hFF, 8'h01, 1'b0); // maximum plus un
113
114    // Cette bulle capture la derniere transaction et vide le pipeline.
115    appliquer_vecteur(1'b0, 8'h00, 8'h00, 1'b0);
116    @(posedge clk_i);
117    #2;
118
119    if (transactions_verifiees !== NOMBRE_TRANSACTIONS) begin
120        $fatal(1, "%0d transactions verifiees, %0d attendues",
121            transactions_verifiees, NOMBRE_TRANSACTIONS);
122    end
123
124    $display("TEST_PASS: adder_pipeline");
125    $finish;
126 end
127
128 endmodule

```

```
1 bash scripts/run_sim.sh adder_pipeline
```

Faux PASS

Un texte contenant le mot PASS ne suffit pas. Le script exige simultanément un code retour Xcelium nul et le marqueur final exact du testbench.

Chapitre 5

Waveforms et méthode de debug

5.1 Batch d’abord, GUI ensuite

Le premier objectif est un run batch reproductible. La GUI sert ensuite à expliquer une divergence, pas à remplacer le verdict du testbench. Le script crée une base SHM et des probes ; l’option `-gui` ouvre la même simulation avec visibilité sur les signaux.

5.2 Lire le full adder

Pour chaque vecteur, vérifier dans cet ordre :

1. les entrées `a_i`, `b_i`, `cin_i` ;
2. la stabilité des entrées avant la comparaison ;
3. `sum_o` ;
4. `cout_o` ;
5. le message final du testbench.

Une sortie `X` dès le temps zéro indique généralement un signal non piloté, un port mal connecté ou un top incorrect.

5.3 Lire le pipeline

Aligner l’analyse sur les fronts montants. Une transaction valide est capturée avec `valid_i` ; le résultat correspondant apparaît un cycle plus tard avec `valid_o`. Le testbench applique les nouvelles entrées au front descendant afin d’éviter une course avec les registres du DUT.

5.4 Ordre de diagnostic

1. lire le premier message d’erreur de compilation ;
2. si la compilation passe, vérifier le nom du top et les ports ;
3. si l’élaboration passe, localiser le premier vecteur en échec ;
4. comparer stimulus, valeur attendue et valeur obtenue ;
5. seulement ensuite ouvrir les waveforms.

À retenir

La waveform explique une erreur ; le testbench la détecte. Conserver cette séparation rend la simulation plus simple à automatiser.

Chapitre 6

Contraintes SDC et analyse temporelle statique

6.1 Le SDC est une spécification

La synthèse ne peut pas deviner quand les entrées arrivent, quelle charge se trouve hors du bloc ou quelle marge doit absorber le jitter. Sans SDC complet, un résultat rapide peut seulement signifier que les chemins importants n'ont pas été analysés.

Le budget de référence utilise une période de 10 ns, soit 100 MHz, un délai d'entrée et de sortie de 1 ns, une incertitude de 0,2 ns, une transition de 0,1 ns et une charge de 0,05 pF. Ces nombres sont des hypothèses pédagogiques documentées, pas des caractéristiques d'un pad ou d'un PCB réel.

6.2 Horloge virtuelle du full adder

Le full adder n'a pas de port clock. Une horloge virtuelle sert uniquement de repère aux délais d'entrée et de sortie. Elle n'apparaît pas dans la netlist et ne doit créer aucun arbre d'horloge physique.

Listing 6.1 – SDC combinatoire avec horloge virtuelle

```
1  # Contraintes pédagogiques du full adder combinatoire.
2  # Unités explicites : 1000 ps = 1 ns et 1000 fF = 1 pF.
3  set_units -time 1000ps
4  set_units -capacitance 1000fF
5
6  set CLOCK_PERIOD_NS      10.000
7  set IO_DELAY_NS          1.000
8  set CLOCK_UNCERTAINTY_NS 0.200
9  set TRANSITION_NS        0.100
10 set OUTPUT_LOAD_PF        0.050
11
12 # Sans port d'horloge, une horloge virtuelle représente l'environnement.
13 create_clock -name vclk -period $CLOCK_PERIOD_NS
14 set_clock_uncertainty $CLOCK_UNCERTAINTY_NS [get_clocks vclk]
15 set_clock_transition  $TRANSITION_NS      [get_clocks vclk]
16
17 set_input_delay  $IO_DELAY_NS -clock vclk [all_inputs]
18 set_output_delay $IO_DELAY_NS -clock vclk [all_outputs]
19 set_input_transition $TRANSITION_NS [all_inputs]
20 set_load $OUTPUT_LOAD_PF [all_outputs]
```

Le chemin budgété va conceptuellement d'un registre externe de lancement à un registre externe de capture. La fenêtre interne disponible dépend des deux délais I/O et de l'incertitude. Modifier la période sans modifier ces hypothèses change la question posée au STA.

6.3 Horloge réelle du pipeline

Le pipeline définit `clk_i` comme source. Les chemins principaux sont entrée-vers-registre, registre-à-registre et registre-vers-sortie. Le reset synchrone est traité comme une entrée fonctionnelle ; aucune fausse exception ne masque son cône logique.

Listing 6.2 – SDC séquentiel à 100 MHz

```

1  # Contraintes pédagogiques de l'additionneur pipeline 8 bits / 100 MHz.
2  # rst_ni est synchrone actif bas : il reste temporisé comme une entrée de
3  # données. Il n'existe volontairement aucun set_false_path sur ce reset.
4  set_units -time 1000ps
5  set_units -capacitance 1000fF
6
7  set CLOCK_PERIOD_NS      10.000
8  set IO_DELAY_NS          1.000
9  set CLOCK_UNCERTAINTY_NS 0.200
10 set TRANSITION_NS         0.100
11 set OUTPUT_LOAD_PF        0.050
12
13 create_clock -name clk -period $CLOCK_PERIOD_NS [get_ports clk_i]
14 set_clock_uncertainty $CLOCK_UNCERTAINTY_NS [get_clocks clk]
15 set_clock_transition $TRANSITION_NS [get_clocks clk]
16
17 set timed_inputs [remove_from_collection [all_inputs] [get_ports clk_i]]
18 set_input_delay $IO_DELAY_NS -clock clk $timed_inputs
19 set_output_delay $IO_DELAY_NS -clock clk [all_outputs]
20 set_input_transition $TRANSITION_NS $timed_inputs
21 set_load $OUTPUT_LOAD_PF [all_outputs]

```

6.4 Setup et hold

Le setup vérifie que la donnée arrive suffisamment avant le prochain front de capture. Une vue lente et des parasites maximaux sont généralement critiques. Le hold vérifie que l'ancienne donnée reste stable après le front courant. Une vue rapide et des parasites minimaux sont généralement critiques. Ces tendances ne dispensent pas de vérifier toutes les vues déclarées.

$$WNS = \min_i (slack_i), \quad (6.1)$$

$$TNS = \sum_{slack_i < 0} slack_i. \quad (6.2)$$

Un PASS pédagogique exige $WNS \geq 0$, $TNS = 0$ et zéro chemin violé. Une seule valeur WNS ne suffit pas : un parseur doit également confirmer que le rapport existe, correspond à la bonne vue et n'est pas tronqué.

6.5 Couverture des contraintes

`check_timing_intent` ou son équivalent doit confirmer les clocks, ports constraints, exceptions et objets résolus. Une commande SDC placée dans un `catch` peut échouer sans arrêter le Tcl ; le flow inspecte donc aussi la collection de commandes SDC échouées et les messages actifs.

Exception dangereuse

Une fausse route ne répare jamais un chemin lent. Elle affirme que ce chemin n'appartient pas au contrat fonctionnel. Toute exception doit nommer sa raison, ses sources, ses destinations et son nombre d'objets attendu.

Chapitre 7

Genus : du Tcl minimal au flow physique

7.1 Niveau 1 — les entrées minimales

Une première synthèse a besoin de quatre éléments :

- le top RTL ;
- une filelist RTL ;
- un SDC ;
- une Liberty typique.

Le script minimal fixe la bibliothèque, lit le RTL, élabore le top, lit le SDC, exécute les contrôles, lance `syn_generic`, `syn_map` et `syn_opt`, puis écrit netlist et SDC mappé.

Listing 7.1 – Tcl Genus minimal en vue typique

```
1  # Synthèse Genus minimale en vue typique.
2  # Le wrapper fournit les chemins par variables d'environnement.
3
4  set flow_dir [file dirname [file normalize [info script]]]
5  source [file join $flow_dir helpers.tcl]
6
7  proc tutorial_minimal_main {} {
8      set root      [tutorial_require_env TUTORIAL_ROOT]
9      set run_dir    [tutorial_require_env GENUS_RUN_DIR]
10     set top        [tutorial_require_env GENUS_TOP_MODULE]
11     set filelist    [tutorial_require_env GENUS_FILELIST]
12     set sdc         [tutorial_require_env GENUS_SDC]
13     set report_dir  [file join $run_dir reports]
14     set output_dir  [file join $run_dir outputs]
15
16     file mkdir $report_dir
17     file mkdir $output_dir
18
19     set_db hdl_language sv
20     set_db library [tutorial_env_list GENUS_LIBERTY_TC]
21     set_db init_hdl_search_path [list $root]
22     cd $root
23
24     tutorial_run_stage read_rtl {
25         read_hdl -sv -f $filelist
26     }
27     tutorial_run_stage elaborate {
28         elaborate $top
29     }
30     tutorial_run_stage read_constraints {
31         read_sdc $sdc
32     }
33     tutorial_run_stage checks {
34         tutorial_report [file join $report_dir check_design.rpt] \
35             {check_design -all}
36         tutorial_report [file join $report_dir check_timing_intent.rpt] \
37             {check_timing_intent -verbose}
```

```

38     }
39     tutorial_run_stage synthesis {
40         syn_generic
41         syn_map
42         syn_opt
43     }
44     tutorial_run_stage reports {
45         tutorial_report [file join $report_dir report_timing.rpt] \
46             {report_timing -max_paths 20}
47         tutorial_report [file join $report_dir report_area.rpt] \
48             {report_area}
49         tutorial_report [file join $report_dir report_qor.rpt] \
50             {report_qor}
51     }
52
53     set mapped_v [file join $output_dir ${top}.mapped.v]
54     set mapped_sdc [file join $output_dir ${top}.mapped.sdc]
55     tutorial_run_stage export {
56         write_hdl > $mapped_v
57         write_sdc > $mapped_sdc
58         tutorial_assert_nonempty $mapped_v
59         tutorial_assert_nonempty $mapped_sdc
60     }
61
62     tutorial_write_final_status PASS \
63         "niveau basic terminé; netlist et SDC disponibles"
64     tutorial_record_stage final_status PASS
65 }
66
67 set rc [catch {tutorial_minimal_main} message options]
68 if {$rc != 0} {
69     catch {tutorial_write_final_status FAIL [tutorial_one_line $message]}
70     puts stderr "GENUS_TUTORIAL_ERROR: $message"
71     if {[dict exists $options -errorinfo]} {
72         puts stderr [dict get $options -errorinfo]
73     }
74     exit 20
75 }
76
77 puts "GENUS_TUTORIAL_STATUS: BASIC_FLOW_COMPLETED"
78 exit 0

```

Après avoir fourni les variables attendues, l'appel direct est :

```

1 genus -files flow/genus/genus_minimal.tcl

```

7.2 Niveau 2 — comprendre les trois transformations

syn_generic transforme le RTL en logique générique et simplifie les expressions.

syn_map choisit des cellules disponibles dans la Liberty.

syn_opt améliore timing, aire et règles électriques selon l'effort demandé.

Une erreur avant **elaborate** concerne souvent les fichiers ou la syntaxe. Une référence non résolue après élaboration concerne la hiérarchie. Un timing mauvais après mapping concerne plutôt contraintes, architecture ou choix de cellules.

7.3 Niveau 3 — utiliser le wrapper

Listing 7.2 – Lanceur Genus basic et advanced

```

1 #!/usr/bin/env bash
2 # Synthèse Genus progressive: niveau basic puis niveau advanced.
3
4 set -u -o pipefail
5
6 usage() {
7     cat <<'EOF'

```

```

8 Usage:
9   LAB_CONFIG=/chemin/prive/lab.env \
10   bash scripts/run_syn.sh --design full_adder_comb
11
12   LAB_CONFIG=/chemin/prive/lab.env \
13   bash scripts/run_syn.sh --design adder_pipeline \
14   --level advanced --view mmmc
15
16 Options:
17   --design full_adder_comb|adder_pipeline
18   --level basic|advanced      (défaut: basic)
19   --view tc|mmmc              (défaut: tc)
20   --run-id IDENTIFIANT
21
22 Variables optionnelles:
23   GENUS_BIN=genus
24   RUN_ROOT=/tmp/digi_tuto_runs
25 EOF
26 }
27
28 fail() {
29   printf 'ERREUR: %s\n' "$*" >&2
30 }
31
32 script_dir=$(CDPATH= cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
33 project_root=$(CDPATH= cd -- "$script_dir/.." && pwd -P)
34
35 design=""
36 level=basic
37 view=tc
38 run_id=${RUN_ID:-}
39
40 while (($# > 0)); do
41   case "$1" in
42     --design)
43       (( $# >= 2 )) || { fail "valeur absente après --design"; exit 2; }
44       design=$2
45       shift 2
46       ;;
47     --level)
48       (( $# >= 2 )) || { fail "valeur absente après --level"; exit 2; }
49       level=$2
50       shift 2
51       ;;
52     --view)
53       (( $# >= 2 )) || { fail "valeur absente après --view"; exit 2; }
54       view=$2
55       shift 2
56       ;;
57     --run-id)
58       (( $# >= 2 )) || { fail "valeur absente après --run-id"; exit 2; }
59       run_id=$2
60       shift 2
61       ;;
62     -h|--help)
63       usage
64       exit 0
65       ;;
66     *)
67       fail "option inconnue: $1"
68       usage >&2
69       exit 2
70       ;;
71   esac
72 done
73
74 case "$design" in
75   full_adder_comb)
76     example_dir=examples/01_full_adder_comb
77     ;;
78   adder_pipeline)
79     example_dir=examples/02_adder_pipeline
80     ;;
81   *)

```

```

82     fail "design invalide: ${design:-<vide>}"
83     usage >&2
84     exit 2
85     ;;
86 esac
87 case "$level" in
88     basic|advanced) ;;
89     *) fail "level doit valoir basic ou advanced"; exit 2 ;;
90 esac
91 case "$view" in
92     tc|mmmc) ;;
93     *) fail "view doit valoir tc ou mmmc"; exit 2 ;;
94 esac
95 if [[ "$level" == basic && "$view" != tc ]]; then
96     fail "le niveau basic utilise uniquement la vue tc"
97     exit 2
98 fi
99
100 design_config="$project_root/$example_dir/syn/design.env"
101 [[ -s "$design_config" ]] || {
102     fail "configuration design absente: $design_config"
103     exit 2
104 }
105 set -a
106 # shellcheck disable=SC1090
107 . "$design_config"
108 lab_config=${LAB_CONFIG:-}
109 if [[ -z "$lab_config" || ! -r "$lab_config" ]]; then
110     fail "LAB_CONFIG doit désigner un fichier privé lisible"
111     exit 2
112 fi
113 # shellcheck disable=SC1090
114 . "$lab_config"
115 set +a
116
117 if [[ -n ${CADENCE_SETUP:-} ]]; then
118     [[ -r "$CADENCE_SETUP" ]] || {
119         fail "initialisation Cadence absente: $CADENCE_SETUP"
120         exit 2
121     }
122     # shellcheck disable=SC1090
123     . "$CADENCE_SETUP"
124 fi
125
126 genus_bin=${GENUS_BIN:-genus}
127 command -v "$genus_bin" >/dev/null 2>&1 || {
128     fail "Genus est introuvable: $genus_bin"
129     exit 127
130 }
131
132 export GENUS_FILELIST="$project_root/$GENUS_FILELIST_REL"
133 export GENUS_SDC="$project_root/$GENUS_SDC_REL"
134 for required in "$GENUS_FILELIST" "$GENUS_SDC"; do
135     [[ -s "$required" ]] || {
136         fail "fichier absent ou vide: $required"
137         exit 2
138     }
139 done
140
141 check_file_list() {
142     local variable_name=$1
143     local raw_value=${!variable_name:-}
144     local item
145     [[ -n "$raw_value" ]] || {
146         fail "variable obligatoire absente: $variable_name"
147         return 1
148     }
149     IFS=: read -r -a values <<< "$raw_value"
150     for item in "${values[@]}"; do
151         [[ -s "$item" ]] || {
152             fail "fichier invalide dans $variable_name: $item"
153             return 1
154         }
155     done

```

```

156 }
157
158 check_file_list GENUS_LIBERTY_TC || exit 2
159 if [[ "$level" == advanced ]]; then
160     check_file_list GENUS_LEF_FILES || exit 2
161     [[ -s ${GENUS_QRC_TC:-} ]] || {
162         fail "GENUS_QRC_TC absent ou vide"
163         exit 2
164     }
165     if [[ "$view" == mmmc ]]; then
166         check_file_list GENUS_LIBERTY_BC || exit 2
167         check_file_list GENUS_LIBERTY_WC || exit 2
168         [[ -s ${GENUS_QRC_BC:-} && -s ${GENUS_QRC_WC:-} ]] || {
169             fail "les fichiers QRC BC et WC sont obligatoires"
170             exit 2
171         }
172     fi
173 fi
174
175 run_root=${RUN_ROOT:-/tmp/digi_tuto_runs}
176 run_id=${run_id:-$(date -u +%Y%m%dT%H%M%SZ)}
177 if [[ ! "$run_id" =~ ^[A-Za-z0-9][A-Za-z0-9._-]*$ ]]; then
178     fail "RUN_ID invalide: $run_id"
179     exit 2
180 fi
181 run_dir="$run_root/genus/$design/$run_id"
182 if [[ -e "$run_dir" ]]; then
183     fail "le dossier existe déjà: $run_dir"
184     exit 2
185 fi
186 mkdir -p "$run_dir/logs" "$run_dir/reports" "$run_dir/outputs"
187 printf 'stage\tstatus\tuttc\tddetail\n' > "$run_dir/stage_status.tsv"
188
189 export TUTORIAL_ROOT="$project_root"
190 export GENUS_RUN_DIR="$run_dir"
191 export GENUS_STAGE_STATUS="$run_dir/stage_status.tsv"
192 export GENUS_MODE="$view"
193 export GENUS_TOP_MODULE
194
195 if [[ "$level" == basic ]]; then
196     flow_tcl="$project_root/flow/genus/genus_minimal.tcl"
197 else
198     flow_tcl="$project_root/flow/genus/genus_main.tcl"
199 fi
200
201 printf 'Synthèse Genus: design=%s level=%s view=%s\n' "$design" "$level" "$view"
202 printf 'Résultats: %s\n' "$run_dir"
203 set +e
204 "$genus_bin" -files "$flow_tcl" -log "$run_dir/logs/genus.log" \
205     2>&1 | tee "$run_dir/logs/console.log"
206 pipeline_rc=("${PIPESTATUS[@]}")
207 set -e
208 tool_rc=${pipeline_rc[0]}
209 tee_rc=${pipeline_rc[1]}
210
211 if ((tool_rc != 0 || tee_rc != 0)); then
212     fail "Genus ou la journalisation a échoué: genus=$tool_rc tee=$tee_rc"
213     exit "$((tool_rc != 0 ? tool_rc : tee_rc))"
214 fi
215 for artifact in \
216     "$run_dir/outputs/$design.mapped.v" \
217     "$run_dir/outputs/$design.mapped.sdc" \
218     "$run_dir/reports/final_status.rpt"; do
219     [[ -s "$artifact" ]] || {
220         fail "sortie obligatoire absente: $artifact"
221         exit 3
222     }
223 done
224 grep -Fqx 'GENUS_STATUS=PASS' "$run_dir/reports/final_status.rpt" || {
225     fail "le statut final Genus n est pas PASS"
226     exit 3
227 }
228
229 printf 'RESULTAT: PASS\n'

```

```

230 printf 'NETLIST=%s\n' "$run_dir/outputs/$design.mapped.v"
231 printf 'SDC=%s\n' "$run_dir/outputs/$design.mapped.sdc"

```

Le niveau basic du full adder s'exécute ainsi :

```

1 export LAB_CONFIG=/chemin/prive/lab.env
2 bash scripts/run_syn.sh --design full_adder_comb

```

Le wrapper vérifie la configuration, la filelist, le SDC et la Liberty avant de consommer une licence. Il place les résultats sous /tmp/digi_tuto_runs/genus/, conserve le code retour et exige trois fichiers non vides : netlist, SDC et statut final.

7.4 Niveau 4 — conscience physique et MMMC

Le niveau advanced ajoute LEF et QRC. Pour le pipeline, il peut construire trois vues : TC nominale, WC pour le setup et BC pour le hold.

Listing 7.3 – Flow Genus avancé par étapes

```

1 # Flow pédagogique Cadence Genus 22.13 : RTL -> netlist mappée.
2 #
3 # Ce script est lancé par scripts/run_syn.sh. Il n'embarque aucun chemin de
4 # laboratoire et ne doit pas être sourcé avant le préflight du wrapper.
5
6 set tutorial_script_dir [file dirname [file normalize [info script]]]
7 source [file join $tutorial_script_dir helpers.tcl]
8
9 proc tutorial_main {} {
10     set root      [tutorial_require_env TUTORIAL_ROOT]
11     set run_dir   [tutorial_require_env GENUS_RUN_DIR]
12     set top       [tutorial_require_env GENUS_TOP_MODULE]
13     set mode      [tutorial_require_env GENUS_MODE]
14     set filelist   [tutorial_require_env GENUS_FILELIST]
15     set mmmc_file [file join $root flow genus mmmc.tcl]
16     set report_dir [file join $run_dir reports]
17     set output_dir [file join $run_dir outputs]
18     foreach dir [list \
19         $report_dir \
20         [file join $report_dir elaboration] \
21         [file join $report_dir timing] \
22         [file join $report_dir qor] \
23         [file join $report_dir messages] \
24         $output_dir] {
25         file mkdir $dir
26     }
27
28     # Réglages volontairement sobres et déterministes pour les deux exemples.
29     set_db source_verbose true
30     set_db hdl_language sv
31     set_db hdl_error_on_latch true
32     set_db detailed_sdc_messages true
33     set_db syn_generic_effort high
34     set_db syn_map_effort high
35     set_db syn_opt_effort high
36
37     cd $root
38
39     tutorial_run_stage read_mmmc {
40         # read_mmmc lit Liberty, QRC et SDC via flow/genus/mmmc.tcl.
41         read_mmmc $mmmc_file
42     }
43
44     tutorial_run_stage read_physical {
45         set tutorial_lefs [tutorial_env_list GENUS_LEF_FILES]
46         read_physical -lef $tutorial_lefs
47     }
48
49     tutorial_run_stage read_rtl {
50         set_db init_hdl_search_path [list $root]
51         read_hdl -sv -f $filelist

```

```

52 }
53
54 tutorial_run_stage elaborate {
55     elaborate $top
56 }
57
58 tutorial_run_stage check_design {
59     tutorial_report \
60         [file join $report_dir elaboration check_design_unresolved.rpt] \
61         {check_design -unresolved}
62     tutorial_report \
63         [file join $report_dir elaboration check_design_all.rpt] \
64         {check_design -all}
65     tutorial_report \
66         [file join $report_dir elaboration report_hierarchy.rpt] \
67         {report_hierarchy}
68 }
69
70 tutorial_run_stage init_design {
71     init_design
72     tutorial_report \
73         [file join $report_dir timing check_timing_intent.rpt] \
74         {check_timing_intent -verbose}
75     tutorial_report \
76         [file join $report_dir timing report_clocks.rpt] \
77         {report_clocks}
78     tutorial_report \
79         [file join $report_dir timing report_timing_pre_synth.rpt] \
80         {report_timing -max_paths 20}
81 }
82
83 tutorial_run_stage syn_generic {
84     syn_generic
85     tutorial_report \
86         [file join $report_dir timing report_timing_post_generic.rpt] \
87         {report_timing -max_paths 20}
88 }
89
90 tutorial_run_stage syn_map {
91     syn_map
92     tutorial_report \
93         [file join $report_dir timing report_timing_post_map.rpt] \
94         {report_timing -max_paths 20}
95 }
96
97 tutorial_run_stage syn_opt {
98     syn_opt
99     tutorial_report \
100         [file join $report_dir timing report_timing_post_opt.rpt] \
101         {report_timing -max_paths 50}
102 }
103
104 tutorial_run_stage reports {
105     tutorial_report [file join $report_dir qor report_qor.rpt] \
106         {report_qor}
107     tutorial_report [file join $report_dir qor report_area.rpt] \
108         {report_area}
109     tutorial_report [file join $report_dir qor report_gates.rpt] \
110         {report_gates}
111     tutorial_report [file join $report_dir qor report_power.rpt] \
112         {report_power}
113     tutorial_report [file join $report_dir qor report_design_rules.rpt] \
114         {report_design_rules}
115     tutorial_report [file join $report_dir messages report_messages.rpt] \
116         {report_messages}
117
118     if {$mode eq "mmc"} {
119         foreach view {tutorial_bc_view tutorial_tc_view tutorial_wc_view} {
120             tutorial_report \
121                 [file join $report_dir timing report_timing_${view}.rpt] \
122                 "report_timing -view $view -max_paths 20"
123         }
124     } else {
125         tutorial_report \

```

```

126         [file join $report_dir timing report_timing_tutorial_tc_view.rpt] \
127         {report_timing -view tutorial_tc_view -max_paths 20}
128     }
129 }
130
131 set mapped_v [file join $output_dir ${top}.mapped.v]
132 set mapped_sdc [file join $output_dir ${top}.mapped.sdc]
133
134 tutorial_run_stage export {
135     write_hdl > $mapped_v
136     if {$mode eq "mmmc"} {
137         # WC est la vue setup canonique du package physique.
138         write_sdc -view tutorial_wc_view > $mapped_sdc
139     } else {
140         write_sdc -view tutorial_tc_view > $mapped_sdc
141     }
142
143     foreach artifact [list $mapped_v $mapped_sdc] {
144         tutorial_assert_nonempty $artifact
145     }
146 }
147
148 tutorial_write_final_status PASS \
149     "niveau advanced terminé; netlist et SDC disponibles; rapports à examiner"
150 tutorial_record_stage final_status PASS
151 }
152
153 set tutorial_rc [catch {tutorial_main} tutorial_error tutorial_options]
154 if {$tutorial_rc != 0} {
155     catch {tutorial_write_final_status FAIL [tutorial_one_line $tutorial_error]}
156     puts stderr "GENUS_TUTORIAL_ERROR: $tutorial_error"
157     if {[dict exists $tutorial_options -errorinfo]} {
158         puts stderr [dict get $tutorial_options -errorinfo]
159     }
160     exit 20
161 }
162
163 puts "GENUS_TUTORIAL_STATUS: ADVANCED_FLOW_COMPLETED_NOT_SIGNOFF"
164 exit 0

```

Listing 7.4 – Création des vues TC et MMMC

```

1  # Définition MMMC portable du tutoriel.
2  #
3  # Toutes les bibliothèques et tous les decks QRC viennent du fichier privé
4  # désigné par LAB_CONFIG. Les listes Liberty utilisent « : » comme séparateur.
5
6  set tutorial_mode [tutorial_require_env GENUS_MODE]
7  set tutorial_sdc [tutorial_require_env GENUS_SDC]
8
9  create_constraint_mode \
10     -name tutorial_functional_mode \
11     -sdc_files [list $tutorial_sdc]
12
13  if {$tutorial_mode eq "tc"} {
14      set tutorial_tc_libs [tutorial_env_list GENUS_LIBERTY_TC]
15      set tutorial_tc_qrc [tutorial_require_env GENUS_QRC_TC]
16      set tutorial_tc_temp [tutorial_require_env GENUS_TEMPERATURE_TC]
17
18      create_rc_corner \
19          -name tutorial_tc_rc \
20          -temperature $tutorial_tc_temp \
21          -qrc_tech $tutorial_tc_qrc
22
23      create_library_set \
24          -name tutorial_tc_libset \
25          -timing $tutorial_tc_libs
26
27      create_timing_condition \
28          -name tutorial_tc_condition \
29          -library_sets tutorial_tc_libset
30
31      create_delay_corner \
32          -name tutorial_tc_corner \

```

```

33     -timing_condition tutorial_tc_condition \
34     -rc_corner tutorial_tc_rc
35
36     create_analysis_view \
37     -name tutorial_tc_view \
38     -constraint_mode tutorial_functional_mode \
39     -delay_corner tutorial_tc_corner
40
41     set_analysis_view \
42     -setup tutorial_tc_view \
43     -hold tutorial_tc_view
44 } elseif {$tutorial_mode eq "mmmc"} {
45     set tutorial_bc_libs [tutorial_env_list GENUS_LIBERTY_BC]
46     set tutorial_tc_libs [tutorial_env_list GENUS_LIBERTY_TC]
47     set tutorial_wc_libs [tutorial_env_list GENUS_LIBERTY_WC]
48
49     foreach corner {bc tc wc} {
50         set upper [string toupper $corner]
51         set qrc [tutorial_require_env GENUS_QRC_${upper}]
52         set temp [tutorial_require_env GENUS_TEMPERATURE_${upper}]
53         create_rc_corner \
54             -name tutorial_${corner}_rc \
55             -temperature $temp \
56             -qrc_tech $qrc
57     }
58
59     create_library_set -name tutorial_bc_libset -timing $tutorial_bc_libs
60     create_library_set -name tutorial_tc_libset -timing $tutorial_tc_libs
61     create_library_set -name tutorial_wc_libset -timing $tutorial_wc_libs
62
63     create_timing_condition \
64         -name tutorial_bc_condition \
65         -library_sets tutorial_bc_libset
66     create_timing_condition \
67         -name tutorial_tc_condition \
68         -library_sets tutorial_tc_libset
69     create_timing_condition \
70         -name tutorial_wc_condition \
71         -library_sets tutorial_wc_libset
72
73     foreach corner {bc tc wc} {
74         create_delay_corner \
75             -name tutorial_${corner}_corner \
76             -timing_condition tutorial_${corner}_condition \
77             -rc_corner tutorial_${corner}_rc
78
79         create_analysis_view \
80             -name tutorial_${corner}_view \
81             -constraint_mode tutorial_functional_mode \
82             -delay_corner tutorial_${corner}_corner
83     }
84
85     # WC pilote l'optimisation setup, BC l'optimisation hold. La vue TC reste
86     # disponible pour corrélation et possède son propre rapport temporel.
87     set_analysis_view \
88         -setup tutorial_wc_view \
89         -hold tutorial_bc_view
90 } else {
91     error "GENUS_MODE invalide: $tutorial_mode (attendu: tc ou mmmc)"
92 }

```

```

1 export LAB_CONFIG=/chemin/prive/lab.env
2 bash scripts/run_syn.sh --design adder_pipeline \
3     --level advanced --view mmmc

```

L'ordre avancé est : MMMC, LEF, RTL, élaboration, contrôles, initialisation du timing, synthèse, rapports puis exports.

7.5 Rapports à lire

1. **check design** : références non résolues, multidrivers, latches ;

2. **timing intent** : clocks et ports non constraints ;
3. **timing** : chemins critiques, WNS et TNS ;
4. **area/QoR** : cellules, aire et qualité globale ;
5. **design rules** : transition, capacitance et fanout.

Fin de commande et qualité

Un code retour nul signifie que le Tcl s'est terminé. Il ne garantit pas que le timing est fermé. La décision finale exige la lecture des rapports.

Chapitre 8

Innovus : de l'import minimal au floorplan contrôlé

8.1 Niveau 1 — fichiers nécessaires

Le premier import Innovus utilise :

- la netlist mappée et le SDC produits par Genus ;
- le technology LEF et le LEF des cellules ;
- une Liberty typique et un fichier QRC ;
- le site des cellules et les noms des pins d'alimentation.

Avant tout lancement long, confirmer que ces fichiers existent, ne sont pas vides et décrivent la même famille technologique.

8.2 Niveau 2 — Tcl minimal

Le script minimal construit une vue TC, initialise le design, connecte les pins PG logiques, crée un floorplan, place, route et exporte DEF et netlist.

Listing 8.1 – Tcl Innovus minimal du full adder

```
1  # Implémentation Innovus minimale du full adder.
2  # Ce niveau montre import, floorplan, placement, routage et export logique.
3
4  proc require_env {name} {
5      if {[info exists ::env($name)] || [string trim $::env($name)] eq ""} {
6          error "variable obligatoire absente: $name"
7      }
8      return $::env($name)
9  }
10
11 proc env_list {name} {
12     set result [list]
13     foreach item [split [require_env $name] ":"] {
14         if {[string trim $item] ne ""} {
15             lappend result [file normalize $item]
16         }
17     }
18     return $result
19 }
20
21 set run_dir [file normalize [require_env PNR_RUN_DIR]]
22 set design [require_env DESIGN]
23 set reports [file join $run_dir reports]
24 set outputs [file join $run_dir outputs]
25 set generated [file join $run_dir generated]
26 set checkpoints [file join $run_dir checkpoints]
27 foreach dir [list $reports $outputs $generated $checkpoints] {
28     file mkdir $dir
29 }
```

```

30
31 set mmmc_file [file join $generated basic_tc.mmmc.tcl]
32 set fh [open $mmmc_file w]
33 puts $fh [list create_constraint_mode -name FUNC -sdc_files [list [require_env PNR_SDC]]]
34 puts $fh [list create_library_set -name LIBSET_TC -timing [env_list LIB_TC]]
35 puts $fh [list create_rc_corner -name RC_TC -temperature [require_env TEMP_TC] \
36     -qx_tech_file [require_env QRC_TECH_FILE_TC]]
37 puts $fh [list create_delay_corner -name CORNER_TC \
38     -library_set LIBSET_TC -rc_corner RC_TC]
39 puts $fh [list create_analysis_view -name TC \
40     -constraint_mode FUNC -delay_corner CORNER_TC]
41 puts $fh [list set_analysis_view -setup [list TC] -hold [list TC]]
42 close $fh
43
44 global init_top_cell init_verilog init_lef_file init_mmmc_file
45 global init_pwr_net init_gnd_net
46 set init_top_cell $design
47 set init_verilog [require_env PNR_NETLIST]
48 set init_lef_file [env_list ALL_LEFS]
49 set init_mmmc_file $mmmc_file
50 set init_pwr_net [require_env POWER_NET]
51 set init_gnd_net [require_env GROUND_NET]
52
53 init_design
54 globalNetConnect $init_pwr_net -type pgpin \
55     -pin [require_env STDCELL_POWER_PIN] -inst * -verbose
56 globalNetConnect $init_gnd_net -type pgpin \
57     -pin [require_env STDCELL_GROUND_PIN] -inst * -verbose
58 applyGlobalNets
59 redirect -file [file join $reports check_design_import.rpt] {checkDesign -all}
60
61 floorPlan -site [require_env STDCELL_SITE] -r \
62     [require_env CORE_ASPECT_RATIO] [require_env CORE_UTILIZATION] \
63     [require_env CORE_MARGIN_UM] [require_env CORE_MARGIN_UM] \
64     [require_env CORE_MARGIN_UM] [require_env CORE_MARGIN_UM]
65 defOut -floorplan [file join $outputs ${design}.floorplan.def]
66
67 placeDesign
68 redirect -file [file join $reports check_place.rpt] {checkPlace}
69
70 setDesignMode -bottomRoutingLayer MET1 -topRoutingLayer MET3
71 routeDesign -placementCheck
72 saveDesign [file join $checkpoints basic_routed.enc]
73
74 set def_path [file join $outputs ${design}.routed.def]
75 set netlist_path [file join $outputs ${design}.routed.v]
76 defOut $def_path
77 saveNetlist $netlist_path
78
79 foreach path [list $def_path $netlist_path] {
80     if {[file exists $path] || [file size $path] == 0} {
81         error "sortie absente ou vide: $path"
82     }
83 }
84
85 set status [open [file join $reports final_status.rpt] w]
86 puts $status "PREFLIGHT_STATUS=PASS"
87 puts $status "IMPORT_STATUS=PASS"
88 puts $status "FLOORPLAN_STATUS=PASS"
89 puts $status "PLACEMENT_STATUS=PASS"
90 puts $status "ROUTE_STATUS=PASS"
91 puts $status "EXPORT_STATUS=PASS"
92 puts $status "SIGNOFF_STATUS=NOT_ACHIEVED"
93 close $status
94
95 puts "DIGI_PNR_STATUS=BASIC_FLOW_COMPLETED"
96 exit 0

```

Lorsque les variables d'environnement sont déjà définies, la commande directe est :

```
1 innovus -no_gui -files flow/innovus/innovus_minimal.tcl
```

Ce niveau est volontairement limité au full adder. Il sert à comprendre la chronologie et ne constitue pas une implémentation physique complète.

8.3 Niveau 3 — préflight et wrapper

Listing 8.2 – Lanceur Innovus basic et advanced

```

1  #!/usr/bin/env bash
2  # Implémentation Innovus progressive: niveau basic puis niveau advanced.
3
4  set -euo pipefail
5
6  usage() {
7      cat <<'EOF'
8  Usage:
9      LAB_CONFIG=/chemin/prive/lab.env \
10     bash scripts/run_pnr.sh validate \
11     DESIGN=full_adder_comb SYN_RUN_DIR=/tmp/.../genus/full_adder_comb/...
12
13     LAB_CONFIG=/chemin/prive/lab.env \
14     bash scripts/run_pnr.sh run \
15     DESIGN=full_adder_comb SYN_RUN_DIR=/tmp/.../genus/full_adder_comb/... \
16     LEVEL=basic
17
18     LAB_CONFIG=/chemin/prive/lab.env \
19     bash scripts/run_pnr.sh run \
20     DESIGN=add_pipeline SYN_RUN_DIR=/tmp/.../genus/add_pipeline/... \
21     LEVEL=advanced VIEW=mmmc
22
23 Variables principales:
24     LEVEL=basic|advanced          (défaut: basic)
25     VIEW=tc|mmmc                 (défaut: tc)
26     RUN_ROOT=/tmp/digi_tuto_runs
27     RUN_ID=nom_du_run
28     INNOVUS_BIN=innovus
29 EOF
30 }
31
32 die() {
33     printf 'ERREUR: %s\n' "$*" >&2
34 }
35
36 mode=${1:-}
37 case "$mode" in
38     validate|run)
39         shift
40         ;;
41     -h|--help|help)
42         usage
43         exit 0
44         ;;
45     *)
46         usage >&2
47         exit 2
48         ;;
49 esac
50
51 for assignment in "$@"; do
52     if [[ $assignment =~ ^[A-Za-z_][A-Za-z0-9_]*=.*$ ]]; then
53         export "$assignment"
54     else
55         die "argument invalide: $assignment"
56         exit 2
57     fi
58 done
59
60 script_dir=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
61 project_root=$(cd -- "$script_dir/.." && pwd -P)
62 export PROJECT_ROOT="$project_root"
63
64 lab_config=${LAB_CONFIG:-}
65 if [[ -z "$lab_config" || ! -r "$lab_config" ]]; then
66     die "LAB_CONFIG doit désigner un fichier privé lisible"
67     exit 2
68 fi
69 # shellcheck source=/dev/null

```

```

70 source "$lab_config"
71 for assignment in "$@"; do
72     export "$assignment"
73 done
74
75 if [[ -n ${CADENCE_SETUP:-} ]]; then
76     [[ -r "$CADENCE_SETUP" ]] || {
77         die "initialisation Cadence absente: $CADENCE_SETUP"
78         exit 2
79     }
80     # shellcheck source=/dev/null
81     source "$CADENCE_SETUP"
82 fi
83
84 level=${LEVEL:-basic}
85 view=${VIEW:-tc}
86 design=${DESIGN:-}
87 syn_run_dir=${SYN_RUN_DIR:-}
88 case "$level" in
89     basic|advanced) ;;
90     *) die "LEVEL doit valoir basic ou advanced"; exit 2 ;;
91 esac
92 case "$view" in
93     tc|mmmc) ;;
94     *) die "VIEW doit valoir tc ou mmmc"; exit 2 ;;
95 esac
96 if [[ -z "$design" || -z "$syn_run_dir" ]]; then
97     die "DESIGN et SYN_RUN_DIR sont obligatoires"
98     exit 2
99 fi
100 syn_run_dir=$(cd -- "$syn_run_dir" 2>/dev/null && pwd -P) || {
101     die "dossier Genus introuvable: $syn_run_dir"
102     exit 2
103 }
104
105 case "$design" in
106     full_adder_comb)
107         export PNR_DESIGN_CONFIG="$project_root/examples/01_full_adder_comb/pnr/design.tcl"
108         has_clock=0
109         ;;
110     adder_pipeline)
111         export PNR_DESIGN_CONFIG="$project_root/examples/02_adder_pipeline/pnr/design.tcl"
112         has_clock=1
113         ;;
114     *)
115         die "design non supporté: $design"
116         exit 2
117         ;;
118 esac
119 if [[ "$level" == basic && "$design" != full_adder_comb ]]; then
120     die "le niveau basic est volontairement limité au full adder"
121     exit 2
122 fi
123 if [[ "$level" == basic && "$view" != tc ]]; then
124     die "le niveau basic utilise uniquement la vue tc"
125     exit 2
126 fi
127
128 export DESIGN="$design"
129 export VIEW="$view"
130 export PNR_NETLIST=${SYN_NETLIST:-$syn_run_dir/outputs/$design.mapped.v}
131 export PNR_SDC=${SYN_SDC:-$syn_run_dir/outputs/$design.mapped.sdc}
132 genus_status="$syn_run_dir/reports/final_status.rpt"
133
134 require_file() {
135     local label=$1
136     local path=$2
137     [[ -s "$path" ]] || {
138         die "$label absent ou vide: $path"
139         return 1
140     }
141 }
142 require_list() {
143     local label=$1

```

```

144     local raw=$2
145     local item
146     [[ -n "$raw" ]] || {
147         die "$label est vide"
148         return 1
149     }
150     IFS=: read -r -a items <<< "$raw"
151     for item in "${items[@]}; do
152         require_file "$label" "$item" || return 1
153     done
154 }
155
156 require_file "netlist Genus" "$PNR_NETLIST" || exit 2
157 require_file "SDC Genus" "$PNR_SDC" || exit 2
158 require_file "statut Genus" "$genus_status" || exit 2
159 grep -Fxq 'GENUS_STATUS=PASS' "$genus_status" || {
160     die "le run Genus fourni n est pas PASS"
161     exit 2
162 }
163
164 export TECH_LEF=${TECH_LEF:-${INNOVUS_TECH_LEF:-}}
165 export STDCELL_LEFS=${STDCELL_LEFS:-${INNOVUS_STDCELL_LEF:-}}
166 export LIB_TC=${LIB_TC:-${INNOVUS_LIBERTY_TC:-}}
167 export LIB_BC=${LIB_BC:-${INNOVUS_LIBERTY_BC:-}}
168 export LIB_WC=${LIB_WC:-${INNOVUS_LIBERTY_WC:-}}
169 export QRC_TECH_FILE_TC=${QRC_TECH_FILE_TC:-${INNOVUS_QRC_TC:-}}
170 export QRC_TECH_FILE_BC=${QRC_TECH_FILE_BC:-${INNOVUS_QRC_BC:-${QRC_TECH_FILE_TC:-}}}
171 export QRC_TECH_FILE_WC=${QRC_TECH_FILE_WC:-${INNOVUS_QRC_WC:-${QRC_TECH_FILE_TC:-}}}
172 export STREAM_MAP=${STREAM_MAP:-${INNOVUS_STREAM_MAP:-}}
173 export STDCELL_GDS=${STDCELL_GDS:-${INNOVUS_STDCELL_GDS:-}}
174 export STDCELL_SITE=${STDCELL_SITE:-}
175 export STDCELL_POWER_PIN=${STDCELL_POWER_PIN:-vddi}
176 export STDCELL_GROUND_PIN=${STDCELL_GROUND_PIN:-gndi}
177 export POWER_NET=${POWER_NET:-VDD}
178 export GROUND_NET=${GROUND_NET:-VSS}
179 export FILLER_CELLS=${FILLER_CELLS:-${INNOVUS_FILLER_CELLS:-}}
180 export CTS_BUFFER_CELLS=${CTS_BUFFER_CELLS:-${INNOVUS_CTS_BUFFERS:-}}
181 export CTS_INVERTER_CELLS=${CTS_INVERTER_CELLS:-${INNOVUS_CTS_INVERTERS:-}}
182 export TIEHI_CELLS=${TIEHI_CELLS:-${INNOVUS_TIE_HIGH_CELLS:-}}
183 export TIELO_CELLS=${TIELO_CELLS:-${INNOVUS_TIE_LOW_CELLS:-}}
184 export ANTENNA_CELLS=${ANTENNA_CELLS:-${INNOVUS_ANTENNA_CELLS:-}}
185 export TIEHI_CELL=""
186 export TIELO_CELL=""
187 export ANTENNA_CELL=""
188 export TEMP_TC=${TEMP_TC:-${GENUS_TEMPERATURE_TC:-25}}
189 export TEMP_BC=${TEMP_BC:-${GENUS_TEMPERATURE_BC:-40}}
190 export TEMP_WC=${TEMP_WC:-${GENUS_TEMPERATURE_WC:-125}}
191
192 require_list TECH_LEF "$TECH_LEF" || exit 2
193 require_list STDCELL_LEFS "$STDCELL_LEFS" || exit 2
194 require_list LIB_TC "$LIB_TC" || exit 2
195 require_file QRC_TECH_FILE_TC "$QRC_TECH_FILE_TC" || exit 2
196 [[ -n "$STDCELL_SITE" ]] || {
197     die "STDCELL_SITE est obligatoire"
198     exit 2
199 }
200
201 if [[ "$level" == advanced ]]; then
202     if [[ "$view" == mmmc ]]; then
203         require_list LIB_BC "$LIB_BC" || exit 2
204         require_list LIB_WC "$LIB_WC" || exit 2
205         require_file QRC_TECH_FILE_BC "$QRC_TECH_FILE_BC" || exit 2
206         require_file QRC_TECH_FILE_WC "$QRC_TECH_FILE_WC" || exit 2
207     fi
208     require_file STREAM_MAP "$STREAM_MAP" || exit 2
209     require_file STDCELL_GDS "$STDCELL_GDS" || exit 2
210     [[ -n "$FILLER_CELLS" ]] || {
211         die "FILLER_CELLS est obligatoire au niveau advanced"
212         exit 2
213     }
214     if ((has_clock == 1)); then
215         [[ -n "$CTS_BUFFER_CELLS" && -n "$CTS_INVERTER_CELLS" ]] || {
216             die "les cellules CTS sont obligatoires pour le pipeline"
217             exit 2

```

```

218     }
219     fi
220 fi
221
222 export ALL_LEFS="$TECH_LEF:$STDCELL_LEFS"
223 export CORE_UTILIZATION=${CORE_UTILIZATION:-0.60}
224 export CORE_ASPECT_RATIO=${CORE_ASPECT_RATIO:-1.0}
225 export CORE_MARGIN_UM=${CORE_MARGIN_UM:-20.0}
226 export MIN_CORE_SIDE_UM=${MIN_CORE_SIDE_UM:-${CORE_MIN_SIDE_UM:-60.0}}
227
228 innovus_bin=${INNOVUS_BIN:-innovus}
229 command -v "$innovus_bin" >/dev/null 2>&1 || {
230     die "Innovus est introuvable: $innovus_bin"
231     exit 127
232 }
233
234 printf 'PNR_PREFLIGHT_STATUS=PASS\n'
235 printf 'DESIGN=%s\nLEVEL=%s\nVIEW=%s\n' "$design" "$level" "$view"
236 if [[ "$mode" == validate ]]; then
237     exit 0
238 fi
239
240 run_root=${RUN_ROOT:-/tmp/digi_tuto_runs}
241 run_id=${RUN_ID:-$(date -u +%Y%m%dT%H%M%SZ)}
242 if [[ ! "$run_id" =~ ^[A-Za-z0-9][A-Za-z0-9-_-]*$ ]]; then
243     die "RUN_ID invalide: $run_id"
244     exit 2
245 fi
246 export PNR_RUN_DIR="$run_root/innovus/$design/$run_id"
247 if [[ -e "$PNR_RUN_DIR" ]]; then
248     die "le dossier existe déjà: $PNR_RUN_DIR"
249     exit 2
250 fi
251 mkdir -p "$PNR_RUN_DIR"/{reports,outputs,checkpoints,generated,logs}
252 {
253     printf 'FLOW=innovus\n'
254     printf 'DESIGN=%s\n' "$design"
255     printf 'LEVEL=%s\n' "$level"
256     printf 'VIEW=%s\n' "$view"
257     printf 'GENUS_RUN_DIR=%s\n' "$syn_run_dir"
258     printf 'SIGNOFF_STATUS=NOT_ACHIEVED\n'
259 } > "$PNR_RUN_DIR/manifest.env"
260
261 if [[ "$level" == basic ]]; then
262     flow_tcl="$project_root/flow/innovus/innovus_minimal.tcl"
263 else
264     flow_tcl="$project_root/flow/innovus/run.tcl"
265 fi
266
267 printf 'Implémentation Innovus: design=%s level=%s view=%s\n' \
268     "$design" "$level" "$view"
269 printf 'Résultats: %s\n' "$PNR_RUN_DIR"
270 set +e
271 "$innovus_bin" -no_gui -files "$flow_tcl" \
272     -log "$PNR_RUN_DIR/logs/innovus.log"
273 run_rc=$?
274 set -e
275 if ((run_rc != 0)); then
276     die "Innovus a échoué: rc=$run_rc"
277     exit "$run_rc"
278 fi
279 grep -Fxq 'EXPORT_STATUS=PASS' "$PNR_RUN_DIR/reports/final_status.rpt" || {
280     die "le statut final Innovus ne contient pas EXPORT_STATUS=PASS"
281     exit 3
282 }
283
284 printf 'RESULTAT: PASS\n'
285 printf 'STATUS_REPORT=%s\n' "$PNR_RUN_DIR/reports/final_status.rpt"
286 printf 'SIGNOFF_STATUS=NOT_ACHIEVED\n'

```

Le préflight peut être exécuté sans lancer Innovus :

```

1 export LAB_CONFIG=/chemin/prive/lab.env
2 bash scripts/run_pnr.sh validate \

```

```

3 DESIGN=full_adder_comb \
4 SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/full_adder_comb/<run>

```

Puis le niveau basic :

```

1 bash scripts/run_pnr.sh run \
2 DESIGN=full_adder_comb \
3 SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/full_adder_comb/<run> \
4 LEVEL=basic

```

Le wrapper vérifie le statut Genus, les vues technologiques et le site avant de créer un dossier sous /tmp/digi_tuto_runs/innovus/.

8.4 Niveau 4 — import avancé et floorplan

Le flow avancé sépare import, floorplan/PG, placement/CTS, route/vérification et exports.

Listing 8.3 – Orchestrateur Innovus avancé

```

1 # Orchestrateur Innovus 22.33. Les erreurs sont conservées dans reports/ et le
2 # processus renvoie un code non nul au wrapper shell.
3
4 if {[info exists ::env(PROJECT_ROOT)] || $::env(PROJECT_ROOT) eq ""} {
5     puts stderr "PNR_ENV_MANQUANTE: PROJECT_ROOT"
6     exit 2
7 }
8 set flow_dir [file join $::env(PROJECT_ROOT) flow innovus]
9 source [file join $flow_dir common.tcl]
10
11 foreach required_dir {reports outputs checkpoints generated logs} {
12     file mkdir [file join [::digi_pnr::require_env PNR_RUN_DIR] $required_dir]
13 }
14
15 source [::digi_pnr::require_env PNR_DESIGN_CONFIG]
16 source [file join $flow_dir 10_import.tcl]
17 source [file join $flow_dir validate.tcl]
18 source [file join $flow_dir 20_floorplan_pg.tcl]
19 source [file join $flow_dir 30_place_cts.tcl]
20 source [file join $flow_dir 40_route_verify_export.tcl]
21
22 ::digi_pnr::initialize_status
23 ::digi_pnr::write_status
24
25 if {[catch {
26     ::digi_pnr::stage import IMPORT_STATUS {
27         ::digi_pnr::import_design
28         ::digi_pnr::validate_loaded_technology
29     }
30     ::digi_pnr::stage floorplan_pg FLOORPLAN_STATUS {
31         ::digi_pnr::run_floorplan_and_pg
32     }
33     ::digi_pnr::stage placement PLACEMENT_STATUS {
34         ::digi_pnr::place_design_and_optimize
35     }
36     ::digi_pnr::stage cts CTS_STATUS {
37         ::digi_pnr::run_cts_if_applicable
38     }
39     ::digi_pnr::stage route_verify_export ROUTE_STATUS {
40         ::digi_pnr::run_route_verify_export
41     }
42 } flow_error flow_options]] {
43     puts stderr $flow_error
44     ::digi_pnr::write_status
45     exit 2
46 }
47
48 ::digi_pnr::write_status
49 puts "DIGI_PNR_STATUS=PASS"
50 puts "DIGI_PNR_FINAL_REPORT=[::digi_pnr::report_path final_status.rpt]"
51 puts "SIGNOFF_STATUS=NOT_ACHIEVED"
52 exit 0

```

Après `init_design`, il contrôle les références, les clocks et la couverture des contraintes. Aucun floorplan n'est accepté si l'import ou le SDC est incomplet.

Pour les très petits exemples, une taille minimale évite un coeur illisible. La densité cible reste 60 % avec un aspect ratio de 1, mais la taille minimale peut dominer ce calcul. Ce cas est pédagogique et doit être signalé, pas caché.

8.5 Pins déterministes

Les entrées sont placées à l'ouest, les sorties à l'est et, pour le pipeline, clock et reset au sud. Chaque port doit posséder une géométrie. Le succès de `editPin` ne suffit pas si une expression de nom ne sélectionne aucun objet.

Chapitre 9

Power planning, lignes et placement

9.1 Deux vocabulaires pour VDD et VSS

Le top logique utilise VDD et VSS. La famille JIHD expose des pins de cellules nommés `vddi` et `gndi`. `globalNetConnect` établit la relation, mais ne dessine aucun métal. Le flow vérifie séparément mapping logique, anneau, rails et connectivité spéciale.

9.2 Anneau et rails

Le profil `xx31` utilise MET3 pour les segments horizontaux de l'anneau et METTP pour les segments verticaux. Les largeurs, espacements et offsets sont des paramètres de l'overlay local et doivent être confirmés avec les règles du PDK. Les rails de cellules sont reliés par `sroute`.

Succès de commande et preuve PG

Une commande `addRing` ou `sroute` réussie ne prouve pas que tous les pins PG sont connectés. La gate finale parse une vérification de connectivité spéciale et compte opens et shorts.

Le guide n'ajoute pas de straps systématiques aux deux petits blocs. L'anneau et les rails suffisent à enseigner la mécanique. Un chapitre futur pourra dimensionner des straps à partir d'un budget de courant et d'une analyse IR/EM.

9.3 Infrastructure de lignes JIHD

Les fillers, ties, diodes antenne et cellules CTS sont validés contre les vues chargées. Aucun tap/endcap CORE JIHD dédié n'est confirmé par le profil de référence. Le flow autorise donc une implémentation pédagogique sous statut `ROW_INFRA_STATUS=PROVISIONAL` ; il interdit toute conclusion de signoff.

Des cellules IO nommées corner ou endcap ne doivent jamais être utilisées pour fermer des lignes CORE par ressemblance de nom. Seule une qualification DRC/LVS du propriétaire technologique peut lever ce statut.

9.4 Placement et optimisation pré-CTS

Le placement global recherche une répartition qui respecte densité, légalité et timing. La légalisation aligne ensuite les cellules sur les sites. Le rapport minimum contient cellules non placées, violations de placement, densité et congestion estimée.

Pour le pipeline, l'optimisation pré-CTS travaille avec une clock idéale. Pour le full adder, ce même stage reste purement combinatoire. Une gate de placement PASS exige zéro instance non placée et zéro violation fatale de légalité, pas seulement une amélioration du WNS.

Chapitre 10

CTS, routage et optimisation post-route

10.1 CTS conditionnel

Le backend interroge les clocks après import. Pour `full_adder_comb`, la seule clock est virtuelle ; aucune pin clock physique n'existe et `CTS_STATUS=NOT_APPLICABLE`. Pour `adder_pipeline`, le CTS cible uniquement `clk_i`.

Les buffers et inverseurs candidats sont fournis par l'overlay puis confirmés dans Liberty et LEF. Le rapport CTS enregistre sinks, cellules insérées, latence, skew et violations. Un arbre construit mais sans sinks résolus est un FAIL.

10.2 Optimisation post-CTS

Après propagation réelle de la clock, les slacks changent. L'optimisation post-CTS traite setup, hold et DRV sans considérer la clock idéale. Les rapports avant/après permettent d'attribuer une dégradation à l'arbre plutôt qu'au datapath.

10.3 Routage des signaux

Les signaux ordinaires sont limités à MET1–MET3. METTP reste disponible pour l'anneau PG et n'est pas promu comme couche signal générale. Le routage global réserve les ressources ; le détail route crée les segments et vias légaux.

10.4 Post-route

Les parasites extraits remplacent l'estimation pré-route. Le flow exécute une optimisation post-route bornée, relit setup et hold, insère les fillers, puis effectue un nettoyage final. L'insertion de fillers peut créer de nouveaux marqueurs ; les vérifications finales sont donc postérieures à cette insertion.

Transcript routeur trompeur

Un résumé de routeur indiquant « DRC 0 » ne remplace pas la commande indépendante `verify_drc`. La gate du tutoriel utilise cette dernière, ainsi que les vérifications de connectivité régulière et spéciale.

10.5 Critères finaux de route

Le PASS exige zéro net non routé, zéro open/short régulier, zéro open/short spécial, zéro marqueur Innovus DRC et zéro violation antenne non justifiée. Une réparation de timing qui casse l'une de ces gates est rejetée.

Chapitre 11

Extraction, STA final et exports

11.1 Extraction RC

Après routage, Innovus extrait résistances et capacités à partir des fichiers QRC associés aux corners. La vérification doit confirmer qu'une analyse post-route a réellement été exécutée.

11.2 STA final

Pour le pipeline, les vues setup et hold actives doivent être relues après extraction. Pour le full adder, la vue TC vérifie le budget combinatoire. Les règles de transition, capacitance et fanout sont également contrôlées, car l'interconnexion réelle peut créer une violation absente en synthèse.

11.3 Livrables du niveau avancé

Livrable	Usage
Checkpoint	Reprendre exactement un stage Innovus.
DEF	Échanger floorplan, placement, pins et routage.
Netlist routée	Intégrer la logique finale dans un niveau supérieur.
SDC routé	Conserver le contrat temporel utilisé.
SPEF	Transmettre les parasites extraits à une analyse autorisée.
LEF abstract	Représenter le bloc comme macro dans un niveau hiérarchique.
GDS	Fournir une géométrie candidate aux vérifications physiques.

L'export GDS utilise la stream map technologique et fusionne le GDS des cellules standard. Ces deux entrées sont fournies par LAB_CONFIG.

11.4 Pourquoi le GDS reste un candidat

Un fichier GDS non vide ne prouve pas le signoff. Les vérifications officielles DRC et LVS, la qualification des cellules de bord, l'analyse IR/EM et la revue de tapeout restent externes à ce tutoriel. Le statut final demeure donc GDS_STATUS=IMPLEMENTATION_CANDIDATE et SIGNOFF_STATUS=NOT_ACHIEVED.

À retenir

Exporter est une opération de format. Qualifier est une opération de preuve. Ne jamais confondre les deux.

Chapitre 12

Gates et diagnostic

12.1 Séparer exécution et qualité

Chaque étape possède deux questions :

1. la commande s'est-elle terminée correctement ?
2. le résultat satisfait-il le critère technique attendu ?

Un placement peut terminer avec des cellules non placées. Un routage peut terminer avec des opens. Un export peut produire un GDS alors que le timing est négatif. Les gates empêchent de transformer ces cas en faux succès.

12.2 Chercher le premier échec causal

Un échec tardif peut être une conséquence. Si l'import contient une référence non résolue, le timing post-route n'est pas une preuve utile. Si le placement n'est pas légal, l'absence d'un rapport de route ne signifie pas que le routeur est la cause.

12.3 Méthode en cinq questions

1. Le top et les fichiers d'entrée sont-ils ceux attendus ?
2. L'outil a-t-il réellement atteint le stage analysé ?
3. Le rapport existe-t-il et correspond-il à la bonne vue ?
4. La métrique lue prouve-t-elle la gate annoncée ?
5. La correction agit-elle sur la cause ou masque-t-elle le symptôme ?

12.4 Ordre de lecture conseillé

Xcelium compilation, élaboration, premier vecteur en échec.

Genus check design, timing intent, timing, règles électriques.

Innovus import, placement, CTS, route, connectivité, DRC, timing, livrables.

À retenir

Le premier rapport négatif fiable est généralement plus utile que le dernier message générique du log.

Chapitre 13

Méthodologie avancée et scripts réutilisables

13.1 La règle d'escalade

On n'ajoute un niveau de complexité que lorsque le niveau précédent est compris et propre :

1. commande directe sur le full adder ;
2. même étape avec un wrapper ;
3. pipeline séquentiel ;
4. vues TC puis MMMC ;
5. contrôles physiques et exports avancés.

Cette règle réduit le nombre de variables simultanées. Si le full adder ne se compile pas, ajouter MMMC ou CTS ne peut pas aider.

13.2 Simulation : simple puis avancée

Commande directe

```
1 xrun -64bit -sv -timescale 1ns/1ps \  
2 -f examples/01_full_adder_comb/sim/rtl.f \  
3 -f examples/01_full_adder_comb/sim/tb.f \  
4 -top tb_full_adder_comb -R
```

Script réutilisable

```
1 bash scripts/run_sim.sh full_adder_comb  
2 bash scripts/run_sim.sh adder_pipeline
```

Le script ajoute isolation des résultats, résolution des filelists, log, waveforms et contrôle du verdict. Les options supplémentaires restent accessibles avec `XRUN_EXTRA_ARGS`.

13.3 Synthèse : logique minimale puis physique

Niveau basic

```
1 export LAB_CONFIG=/chemin/prive/lab.env  
2 bash scripts/run_syn.sh --design full_adder_comb
```

Ce niveau utilise une Liberty TC, le RTL et le SDC.

Niveau advanced

```
1 bash scripts/run_syn.sh --design adder_pipeline \
2   --level advanced --view mmmc
```

Ce niveau ajoute LEF, QRC, plusieurs bibliothèques, rapports par vue et contrôles de timing plus complets.

13.4 Implémentation : démonstration puis flow complet

Niveau basic

```
1 bash scripts/run_pnr.sh run \
2   DESIGN=full_adder_comb \
3   SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/full_adder_comb/<run> \
4   LEVEL=basic
```

Le full adder montre import, floorplan, placement, route et exports simples.

Niveau advanced

```
1 bash scripts/run_pnr.sh run \
2   DESIGN=adder_pipeline \
3   SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/adder_pipeline/<run> \
4   LEVEL=advanced VIEW=mmmc
```

Le pipeline ajoute power planning, cellules physiques, CTS, optimisation post-route, connectivité, DRC, extraction, SPEF, LEF abstract et GDS candidat.

13.5 Architecture recommandée des scripts

Un flow maintenable sépare trois responsabilités :

Wrapper shell paramètres utilisateur, validation des chemins, dossier de résultats et code retour.

Tcl outil commandes Genus ou Innovus organisées par stages.

Fichiers design top, filelist, SDC et politique physique propres à chaque exemple.

Lorsqu'un nouveau bloc est ajouté, commencer par copier uniquement les fichiers design. Les scripts génériques ne doivent changer que si la méthode elle-même évolue.

13.6 Critère d'arrêt

Le niveau avancé n'est pas synonyme de signoff. Arrêter et diagnostiquer dès qu'une gate essentielle échoue : top non résolu, port non contraint, slack négatif, cellule non placée, net non routé, connectivité PG incorrecte ou DRC non nul.

Complexité utile

Ajouter une option n'est justifié que si elle répond à une question identifiable et si un rapport permet de vérifier son effet.

Annexe A

Commandes de référence

A.1 Simulation Xcelium

```
1 # Commande minimale
2 xrun -64bit -sv -timescale 1ns/1ps \
3   -f examples/01_full_adder_comb/sim/rtl.f \
4   -f examples/01_full_adder_comb/sim/tb.f \
5   -top tb_full_adder_comb -R
6
7 # Scripts
8 bash scripts/run_sim.sh full_adder_comb
9 bash scripts/run_sim.sh adder_pipeline
```

A.2 Synthèse Genus

```
1 export LAB_CONFIG=/chemin/prive/lab.env
2
3 # Niveau basic TC
4 bash scripts/run_syn.sh --design full_adder_comb
5
6 # Niveau advanced MMMC
7 bash scripts/run_syn.sh --design adder_pipeline \
8   --level advanced --view mmmc
```

A.3 Implémentation Innovus

```
1 export LAB_CONFIG=/chemin/prive/lab.env
2
3 # Vérifier les entrées sans lancer le flow
4 bash scripts/run_pnr.sh validate \
5   DESIGN=full_adder_comb \
6   SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/full_adder_comb/<run>
7
8 # Niveau basic
9 bash scripts/run_pnr.sh run \
10  DESIGN=full_adder_comb \
11  SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/full_adder_comb/<run> \
12  LEVEL=basic
13
14 # Niveau advanced
15 bash scripts/run_pnr.sh run \
16  DESIGN=adder_pipeline \
17  SYN_RUN_DIR=/tmp/digi_tuto_runs/genus/adder_pipeline/<run> \
18  LEVEL=advanced VIEW=mmmc
```

A.4 Inspection après un run

Commencer par le statut final, puis lire le premier rapport associé à une gate négative. Le log complet sert ensuite à retrouver la commande ou le message ayant produit ce résultat.

Annexe B

Erreurs courantes

Symptôme	Cause typique	Diagnostic	Action
xrun introuvable	environnement Cadence non chargé	<code>command -v xrun</code>	Charger le setup autorisé.
RTL absent	chemin relatif au mauvais dossier	ouvrir la filelist résolue	Utiliser le dossier principal ou un chemin absolu.
Top inconnu	mauvais nom avec <code>-top</code>	relire le nom du module testbench	Corriger le top.
Marqueur PASS absent	test interrompu ou <code>\$fatal</code>	lire le premier vecteur en échec	Corriger DUT ou testbench.
Genus termine sans netlist	Tcl interrompu ou mauvais dossier de sortie	lire <code>stage_status.tsv</code>	Corriger le premier stage FAIL.
Référence non résolue	module absent de la filelist	lire <code>check_design</code>	Ajouter ou corriger le fichier RTL.
WNS positif mais ports libres	contraintes incomplètes	lire timing intent	Compléter le SDC.
CTS sans sinks	clock virtuelle ou mauvais port	lire clocks et rapport CTS	Marquer N/A ou corriger la source.
Placement incomplet	site, floorplan ou cellules incompatibles	lire <code>checkPlace</code>	Corriger la première violation.
Commande PG réussie mais opens présents	métal ou cible non connectés	<code>verifyConnectivity -type special</code>	Corriger la géométrie PG.
Route terminée mais DRC non nul	transcript insuffisant	<code>verify_drc</code>	Réparer puis relancer la vérification.
GDS présent sans LVS	export confondu avec qualification	vérifier les statuts physiques	Garder le statut candidat.

Annexe C

Glossaire

BC Best Case, corner rapide souvent critique pour le hold.
CTS Clock Tree Synthesis, construction physique de l'arbre d'horloge.
DEF Description physique d'un design placé ou routé.
DRV Design Rule Violation : transition, capacitance, fanout, etc.
GDS Géométrie de layout remise aux vérifications physiques.
LEF Vue géométrique abstraite de technologie ou de cellule.
Liberty Modèle timing, puissance et règles électriques d'une cellule.
MMMC Multi-Mode Multi-Corner, ensemble cohérent de vues d'analyse.
OOO Out Of Context, bloc contraint hors de son environnement complet.
PG Power/Ground, réseaux d'alimentation et de masse.
QRC Modèle d'extraction résistances/capacités par corner.
SDC Synopsys Design Constraints, contrat de timing.
SPEF Standard Parasitic Exchange Format, parasites extraits.
STA Static Timing Analysis, analyse temporelle sans vecteurs.
TC Typical Case, vue nominale pédagogique.
TNS Somme des slacks négatifs.
WC Worst Case, corner lent souvent critique pour le setup.
WNS Slack le plus négatif, ou marge minimale si positif.

Références

- [1] *Xcelium Logic Simulation — Command-Line and Tcl Reference*. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems.
- [2] *Genus Synthesis Solution — User Guide and Command Reference*. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems.
- [3] *Innovus Implementation System — User Guide and Command Reference*. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems.
- [4] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*. IEEE 1800-2017. IEEE. 2018.